

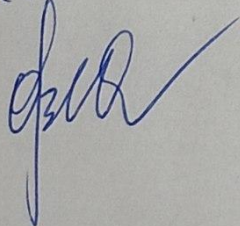
Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К курсовой работе

по курсу «Программирование на языке Java»

на тему «Разработка многомодульного приложения на языке Java»

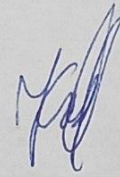
Отлично
21.05.26


Выполнил:
Студент группы 23ВВП1
А.П. Володин
Приняла:
О.В. Юрова

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет Вычислительной техники

Кафедра "Вычислительная техника"



"УТВЕРЖДАЮ"

Зав. кафедрой ВТ

профессор М. А. Митрохин

«___» _____ 20__ г

ЗАДАНИЕ

на курсовое проектирование по курсу

Программирование на языке Java
Студенту Володину А.П. Группа 23ВВТ1
Тема проекта Разработка многомодульного приложения на языке Java

Исходные данные (технические требования) на проектирование

Разработать клиент-серверное приложение "Текстовый чат".
Язык программирования Java, среда разработки NetBeans.
Приложение должно обладать графическим интерфейсом и использовать следующие технологии:

1. Java Collections Framework.
2. Механизм обработки исключительных ситуаций
3. Java Stream API
4. Java Multithreading
5. Сетевое взаимодействие

Объем работы по курсу

1. Расчетная часть

2. Графическая часть

Диаграммы языка UML: диаграмма вариантов использования;
диаграмма классов;
диаграмма деятельности;
диаграмма развертывания;
диаграмма последовательности.

3. Экспериментальная часть

Тестирование многомодульного приложения.

Срок выполнения проекта по разделам

1 В соответствии с графиком.

2

3

4

5

6

7

8

Дата выдачи задания “ ”

Дата защиты проекта “ ”

Руководитель Корова В.В.

Задание получил “ 24 ” февраля

2026 г.

Студент Володик Артём Павлович

Содержание

Введение.....	5
1. Постановка задачи	6
2. Выбор решения	7
3. Описание программы.....	9
4. Описание способа организации пользовательского интерфейса	17
5. Описание результатов работы программы	18
Заключение.....	22
Список литературы.....	23
Приложение А. Листинг программы.....	24
Приложение Б. UML – диаграммы	53

Введение

Современные условия делают средства коммуникации неотъемлемым элементом повседневной жизни. Данный курсовой проект направлен на разработку мессенджера – приложения, которое позволит пользователям в пределах локальной сети обмениваться текстовыми сообщениями и файлами. Основная цель работы заключается в создании удобного инструмента для живого общения и оперативной передачи данных в реальном времени.

В рамках разработки предстоит решить ряд важных задач, включая организацию многопользовательского доступа, внедрение системы авторизации и аутентификации, а также реализацию механизма управления файлами. Процесс будет освещен комплексно: от этапа проектирования общей структуры программного обеспечения до непосредственного воплощения его ключевых функций.

В итоге получится приложение, которое обеспечит комфортное взаимодействие между пользователями и эффективный обмен файлами в локальной сетевой среде. Приобретенный в ходе работы опыт и полученные результаты могут послужить основой для дальнейшего изучения принципов создания подобных систем и их последующего развития и улучшения.

1. Постановка задачи

Цель работы заключается в разработке клиент-серверной архитектуры, предназначенной для обмена текстовыми сообщениями в локальной вычислительной сети. Необходимо создать систему, в которой клиенты могут взаимодействовать с сервером на основе вновь создаваемого протокола обмена данными. Необходимой частью является наличие графического интерфейса пользовательского приложения.

Требуется разработать протокол обмена данными, который обеспечит передачу информации между клиентами и сервером. Графический интерфейс пользовательского приложения должен поддерживать отправку как индивидуальных, так и общих сообщений.

Так же дополнительно система может предоставлять возможности идентификации и авторизации пользователей, что повышает уровень безопасности и обеспечивает защищенный доступ к обмену данными.

Для реализации поставленной задачи выбран язык программирования Java как кроссплатформенное объектно-ориентированное решение. В качестве среды разработки используется Apache NetBeans, обеспечивающая удобное управление модулями проекта и встроенную поддержку графического конструктора форм. Основными технологиями выступают: фреймворк Java Swing для построения графического интерфейса, классы `java.net` для работы с TCP/IP-сокетами, многопоточная модель (`java.lang.Thread`) для параллельной обработки подключений, а также механизмы сериализации сообщений и кодирования бинарных данных в формат Base64 [1-5].

2. Выбор решения

Для создания графического интерфейса клиентского приложения был использован фреймворк Java Swing [2]. В качестве основного контейнера окна применён класс JFrame, с помощью которого в интерфейс интегрированы стандартные компоненты: текстовые поля, кнопки и списки.

Организация взаимодействия между серверной и клиентской частями системы построена на основе протокола TCP/IP [5]. Этот протокол гарантирует надежную передачу данных за счет использования механизмов подтверждения получения информации и повторной отправки в случае сбоев. Широкое распространение TCP/IP подтверждается его применением в таких прикладных протоколах, как HTTP, FTP, SMTP и Telnet. Установка соединения по TCP требует предварительного согласования: сервер ожидает входящие подключения, а клиент инициирует соединение, отправляя синхронизирующий порядковый номер. В разрабатываемой системе реализована топология "звезда", где все клиенты подключаются к центральному узлу - серверу. Наглядное представление данной архитектуры показано на рисунке 1.

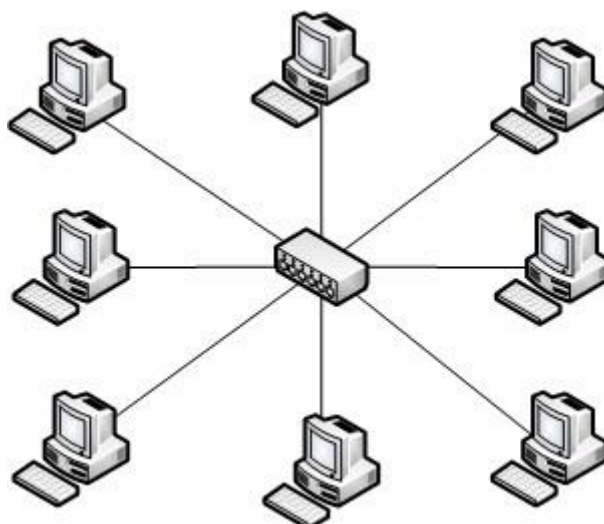


Рисунок 1 – Топология “звезда”

Разработка велась с использованием модульной архитектуры, разделяющей систему на клиентскую и серверную части [1]. Серверная часть включает три основных модуля: класс `Server`, отвечающий за приём входящих подключений и запуск обработчиков; класс `ClientHandler`, выполняющийся в отдельном потоке для каждого подключённого клиента и обрабатывающий входящие команды; класс `UserManager`, управляющий базой учётных записей и выполняющий регистрацию и аутентификацию пользователей. Клиентская часть состоит из четырёх модулей: класс `Main`, являющийся точкой входа в приложение; класс `ConnectionForm1`, отвечающий за ввод сетевых параметров и проверку доступности сервера; класс `AuthForm`, реализующий формы входа и регистрации с валидацией вводимых данных; класс `ChatForm`, управляющий основным окном чата, обработкой сетевых сообщений, отправкой файлов и ведением локальных логов переписки.

Объектно-ориентированная структура системы и взаимосвязи между описанными модулями визуализированы на UML-диаграмме классов (см. приложение Б, рисунок 24). Диаграмма отражает ключевые классы серверной и клиентской частей, их атрибуты, публичные методы, а также типы отношений (композиция, агрегация, зависимость и реализация интерфейсов). Такая структура обеспечивает слабую связанность компонентов, инкапсуляцию бизнес-логики и возможность независимого масштабирования клиентского и серверного модулей.

Для обеспечения безопасного обмена данными между клиентами и сервером в решение был внедрен механизм идентификации и авторизации пользователей. Это обеспечивает контроль доступа к системе и защиту от стороннего использования.

3. Описание программы

Программа разработана на языке программирования Java в среде Apache NetBeans [1]. Клиентская часть построена с использованием фреймворка Java Swing, обеспечивающего кроссплатформенность и стандартный внешний вид окон [2]. Сетевое взаимодействие организовано на основе TCP/IP-сокетов, а многопоточная модель позволяет серверу параллельно обрабатывать подключения нескольких клиентов [4].

Необходимые входные данные для работы системы:

- Сетевые параметры: IP-адрес сервера (IPv4 или localhost) и порт (1–65535);
- Учётные данные: логин (3–20 символов, латиница/цифры/подчёркивание) и пароль (6–30 символов);
- Текстовые сообщения для отправки;
- Файлы для передачи (размером до ~48 КБ в кодированном виде).

Принцип работы приложения:

1. Сервер запускается и переходит в режим ожидания входящих соединений на заданном порту.
2. Клиент вводит параметры сети, система проверяет доступность узла и открывает форму авторизации.
3. После успешной аутентификации сервер присваивает клиенту идентификатор, запускает фоновые таймеры поддержания соединения и отображает главное окно чата.
4. Сообщения и файлы маршрутизируются сервером согласно типу команды протокола (личное/широковещательное/файл).
5. При закрытии окна или разрыве соединения клиент корректно освобождает сетевые ресурсы и завершает рабочие потоки.

Визуальное представление алгоритма работы системы в виде UML-диаграммы деятельности представлено в приложении Б (рисунок 25). Диаграмма отражает параллельные процессы на стороне сервера и клиента, включая обработку событий, принятие решений и передачу управления между компонентами.

Ниже представлена работа клиент-серверного приложения. Для начала

необходимо запустить сервер (рисунок 2).



Рисунок 2 – Запуск сервера.

Далее необходимо запустить приложение клиента. При запуске приложения откроется окно ввода IP адреса и порта (рисунок 3). Если подключение успешно, то откроется форма авторизации и аутентификации. Если подключение не удалось, то пользователя уведомят об этом, выведя информацию об ошибке подключения.

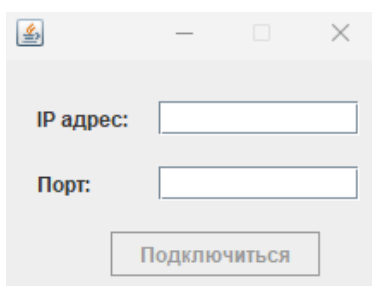


Рисунок 3 – Подключение к серверу.

После успешного подключения пользователю предложат пройти процесс авторизации и аутентификации при помощи формы на рисунке 4. В этой форме необходимо указать логин и пароль и выбрать необходимое действие – войти или зарегистрироваться.

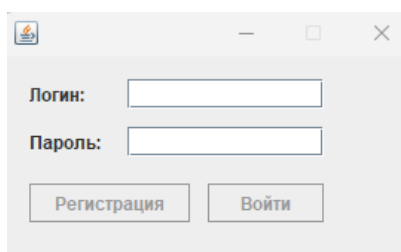


Рисунок 4 – Авторизация и аутентификация.

В процессе авторизации сервер обрабатывает данные пользователя, разрешая или запрещая дальнейшую деятельность в приложении. В процессе работы сервер выводит результаты обработки команд в консоль (рисунок 5).

Клиент подключён: 127.0.0.1

Пользователь 'artem1' вошёл с IP: 127.0.0.1

Рисунок 5 – Процесс работы сервера.

В данном случае сервер принял команду *Login* и ответил пользователю, что вход разрешён. Сразу же после этого подключенный клиент начал опрашивать сервер о пользователях при помощи команды *UsersListUpdate*.

После успешной процедуры аутентификации пользователю открывается основное окно мессенджера, представленная на рисунке 6.

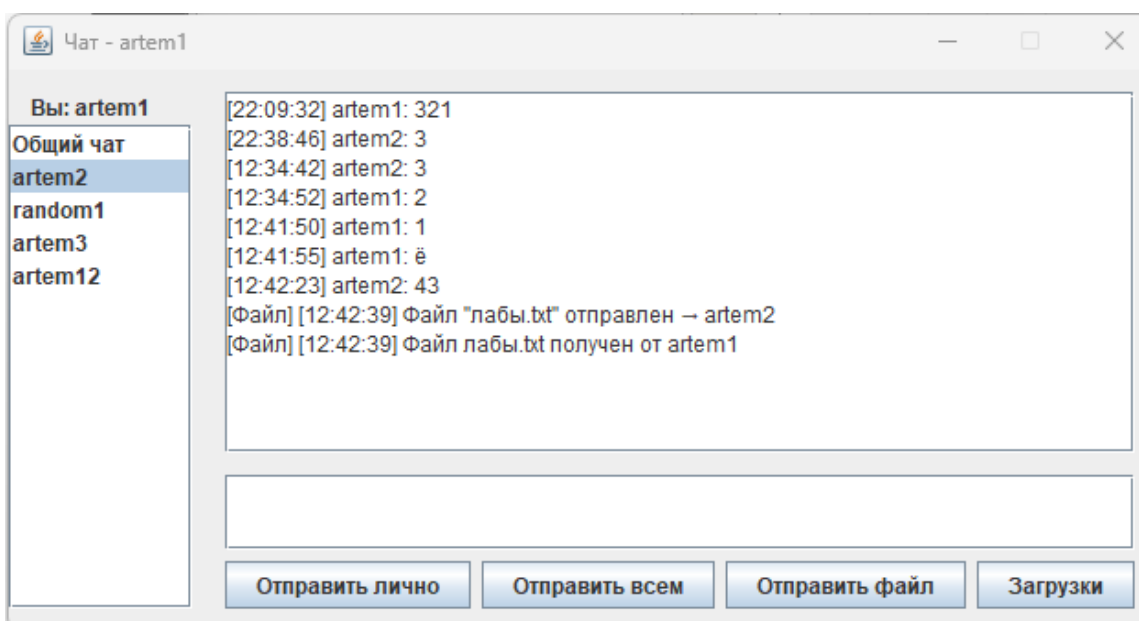


Рисунок 6 – Главное окно.

Для того, чтобы начать обмен сообщениями, необходимо выбрать пользователя, которому необходимо отправить сообщение, напечатать само сообщение и нажать кнопку “Отправить лично”.

После отправки сообщения оно отобразится в панели, как показано на рисунке 7. Пример того, как это сообщение отобразится у получателя представлен на рисунке 8.

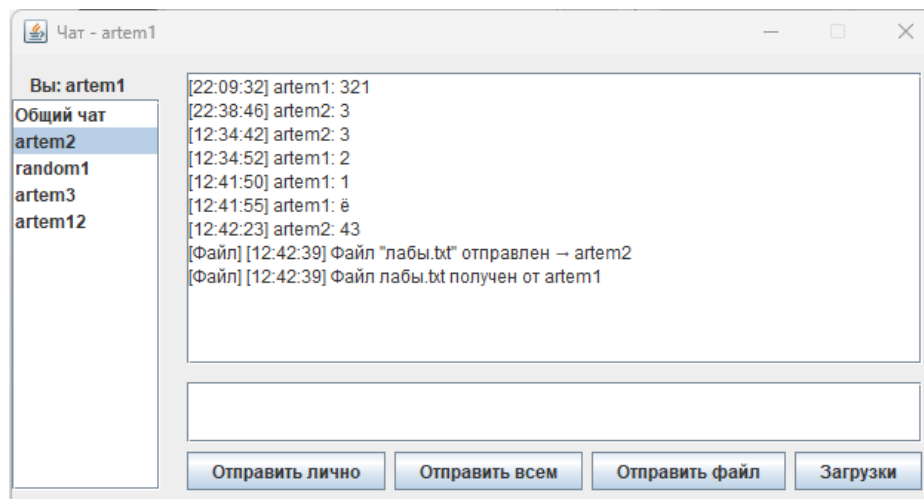


Рисунок 7 – Отправка личного сообщения.

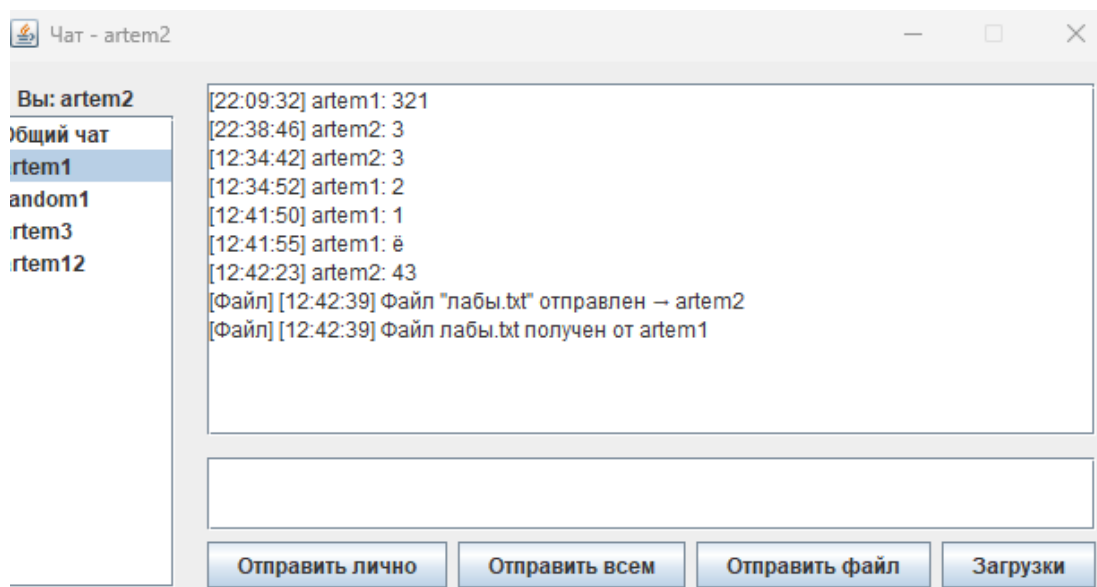


Рисунок 8 – Получение личного сообщения.

Стоит отметить, что при выборе разных пользователей в панели будут отображаться истории сообщений с этими пользователями. Это достигается через хранение переписок у каждого пользователя индивидуально.

Если нужно отправить общее сообщение, то необходимо набрать это сообщение и нажать кнопку “Отправить всем” и перейти в «Общий чат», в котором хранятся все общие сообщения. Сообщение отобразится у пользователя отправителя, как показано на рисунке 9. Для других получателей оно будет отражено как показано на рисунке 10.

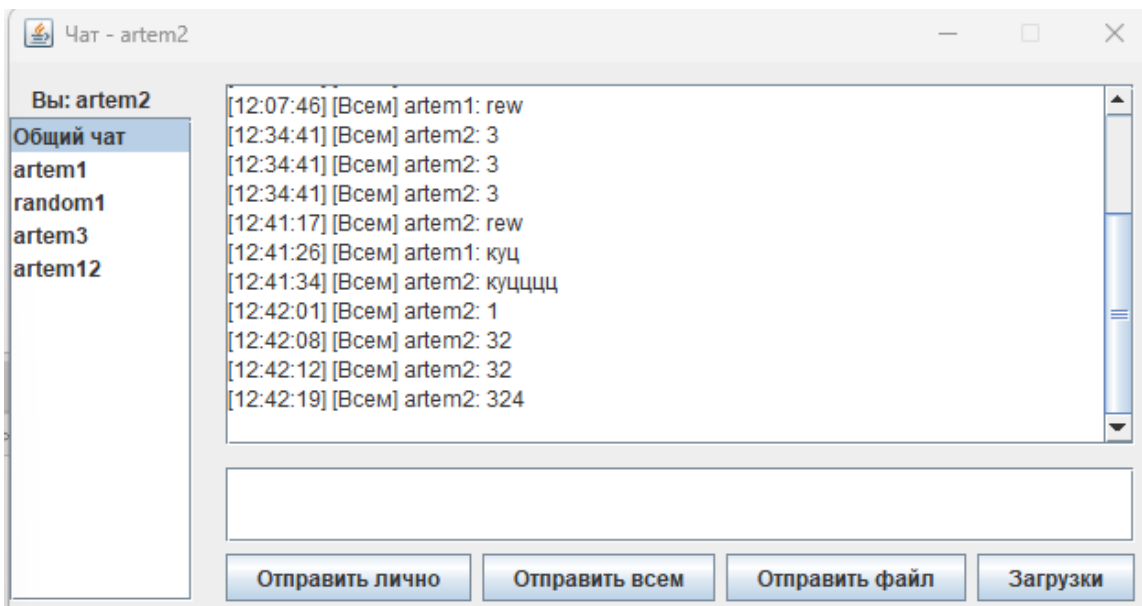


Рисунок 9 – Отправка широковещательного сообщения.

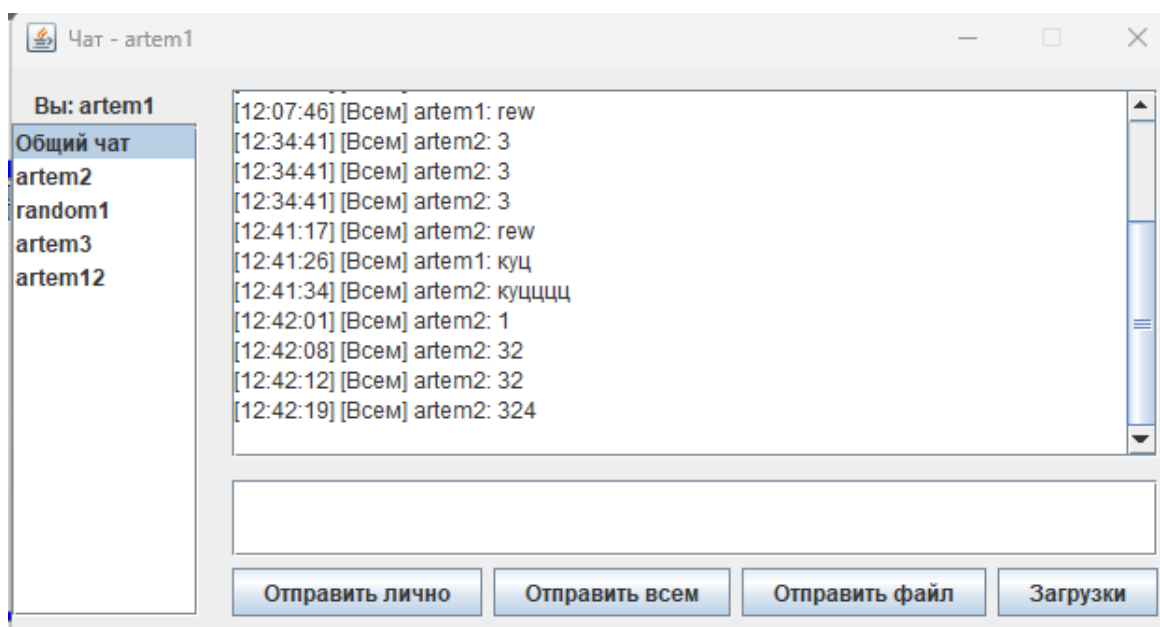


Рисунок 10 – Получение общего сообщения.

Для того, чтобы отправить файл необходимо выделить получателя в панели и нажать на кнопку “Выбрать файлы”. После этого откроется окно выбора файлов (рисунок 11).

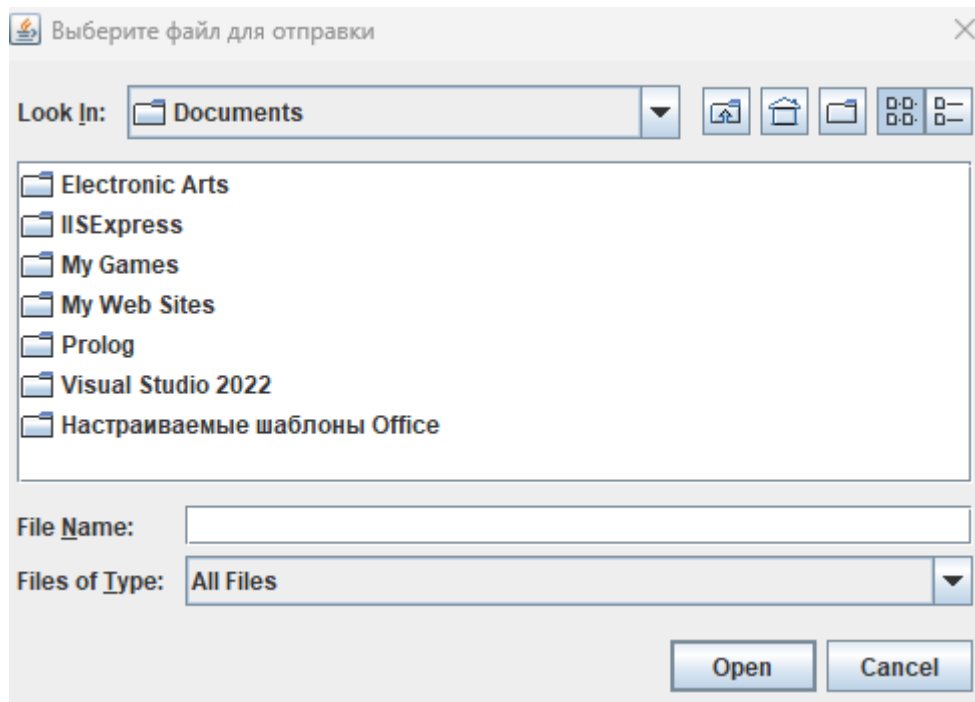


Рисунок 11 – Выбор файла для отправки.

После выбора файла автоматически запустится его отправка выбранному клиенту. Так же у выбранного клиента сначала появится сообщение, что ему от клиента отправителя направлен файл, а по факту полной загрузки файла клиенту получателю будет выведено сообщение, о том, что файл отправлен, как показано на рисунке 12.

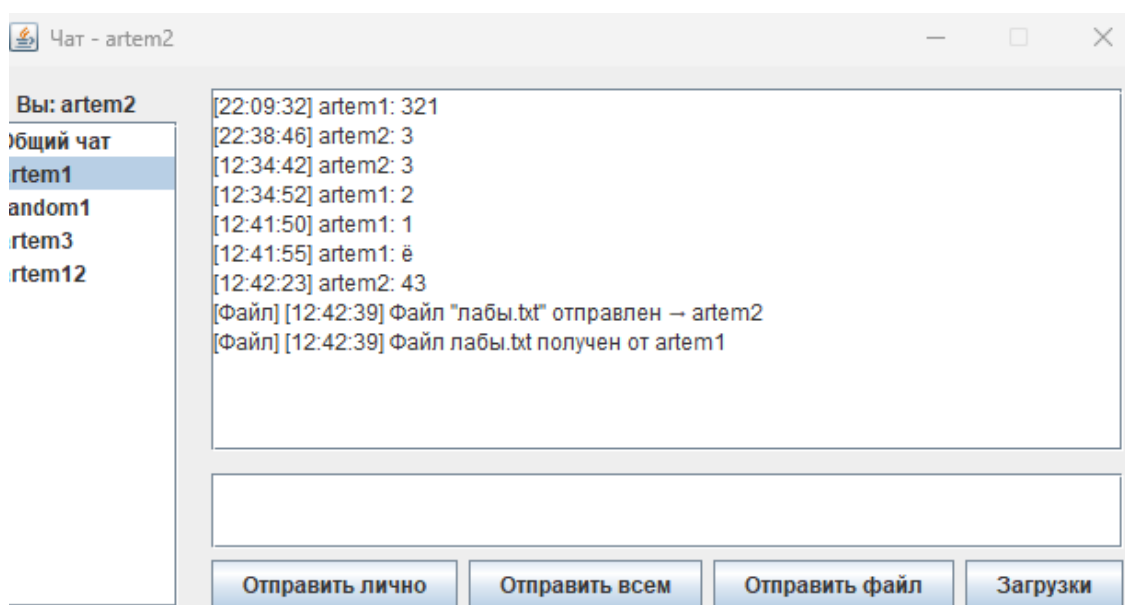


Рисунок 12 – Принятие файла.

Если нужно открыть полученный файл, то необходимо нажать кнопку “Загрузки”. После этого откроется папка с файлом, как показано на рисунке 13.

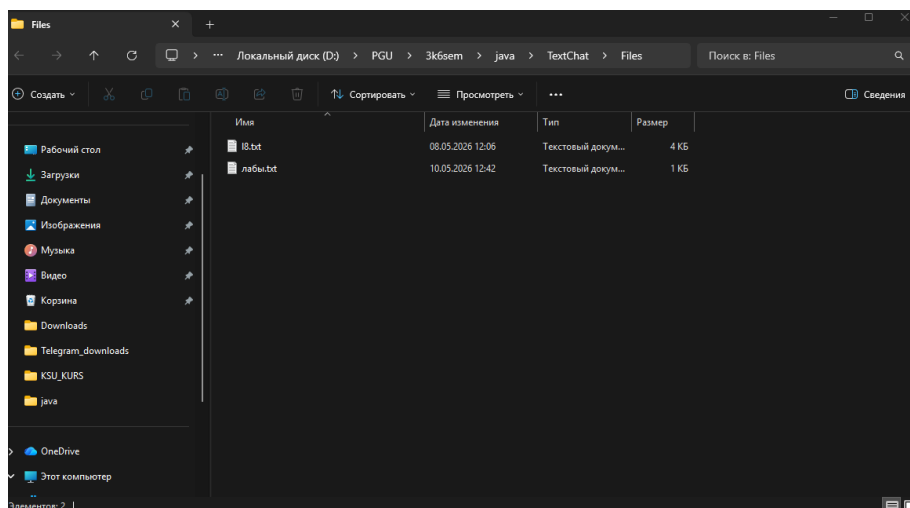


Рисунок 13 – Папка с файлами.

Для наглядного понимания функционала, которое может предоставить приложения была создана UML-диаграмма вариантов использования для клиента, которая представлена на рисунке 14.

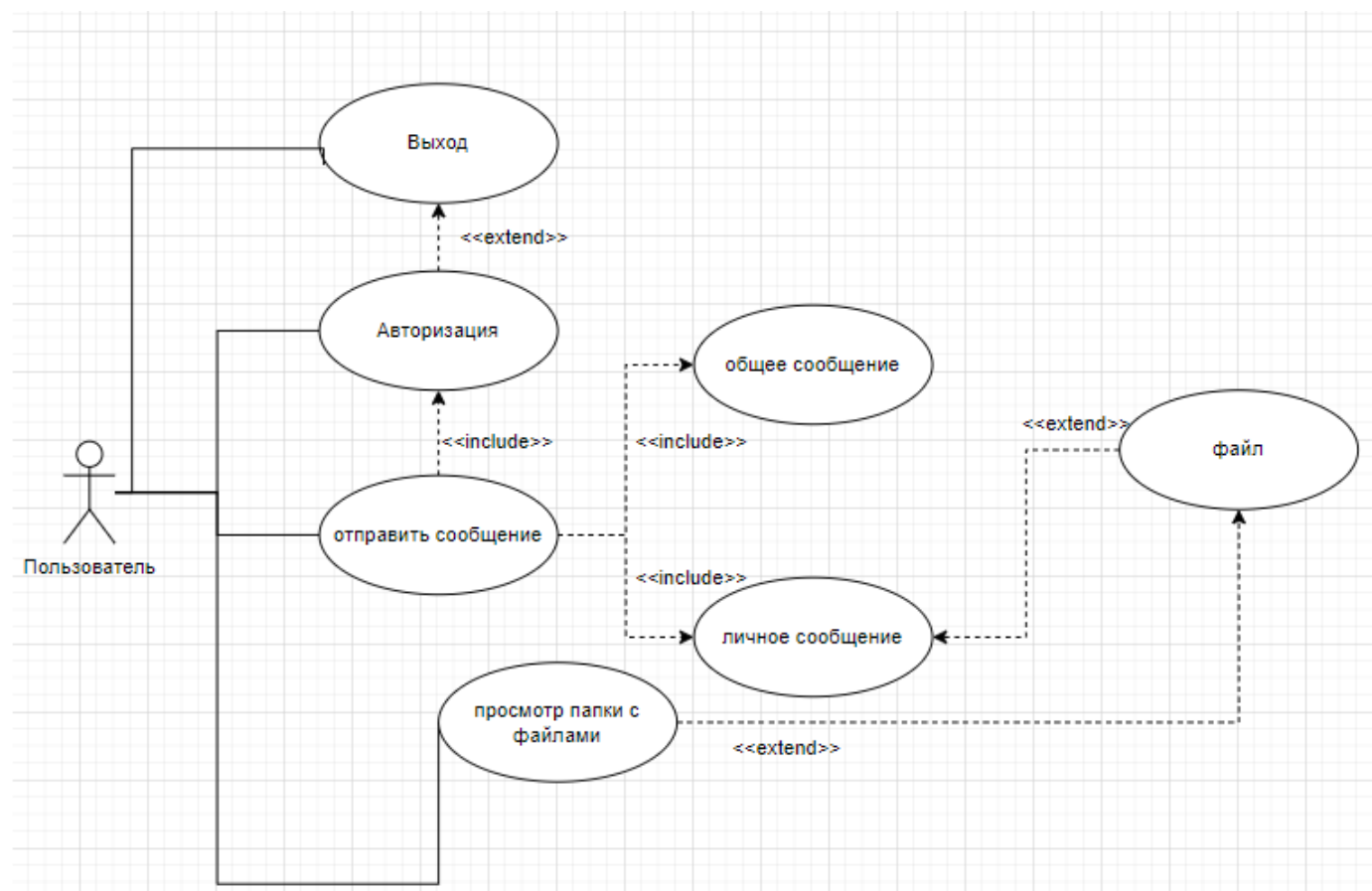


Рисунок 14 – UML-диаграмма вариантов использования.

Для выделения основных взаимодействий между клиентской и серверной частями приложения, была разработана UML-диаграмма развертывания. Она служит для визуализации физической структуры системы и взаимодействия её компонентов. На рисунке 15 представлена данная диаграмма, где показана клиентская и серверная часть приложений, а также способы их взаимодействия.

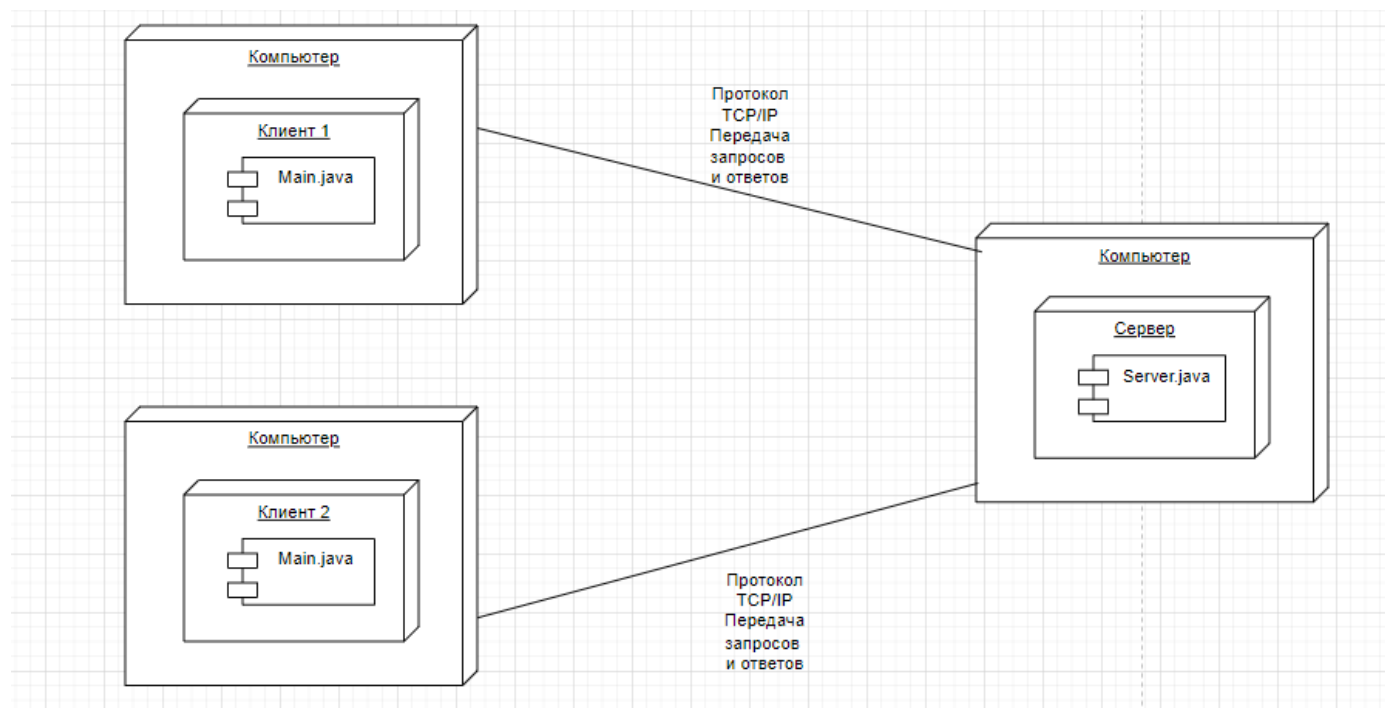


Рисунок 15 – UML - диаграмма развёртывания.

4. Описание способа организации пользовательского интерфейса

Графический интерфейс клиентского приложения реализован с использованием библиотеки Java Swing, основанной на компонентно-ориентированной модели. В качестве главного контейнера окон применяется класс JFrame, внутри которого размещаются панели с менеджерами компоновки GroupLayout, обеспечивающими точное выравнивание элементов и стабильное отображение при изменении размеров окна [2].

Ключевые компоненты интерфейса и их назначение:

- JList с моделью DefaultListModel – динамический список активных пользователей и пункт «Общий чат». Обновляется без перерисовки всего окна за счёт привязки к модели данных.

- JTextArea в контейнере JScrollPane – область вывода истории сообщений. Флаг setEditable(false) блокирует ручное редактирование, а программная автопрокрутка гарантирует отображение новых строк.

- JTextField и JPasswordField – поля ввода текста и пароля (с маскировкой символов setEchoChar('*')).

- JButton – кнопки управления отправкой сообщений, файлов и навигацией. Их состояние (активно/неактивно) управляется через DocumentListener, отслеживающий валидность введённых данных в реальном времени.

Организация многопоточности и событий:

Для предотвращения блокировки интерфейса сетевое чтение данных вынесено в отдельный фоновый поток (Thread) [4]. Все обновления компонентов GUI делегируются в главный поток обработки событий через SwingUtilities.invokeLater(), что гарантирует потокобезопасность. Валидация ввода и обработка действий пользователя реализованы через стандартные слушатели событий (ActionListener, ListSelectionListener), а системные уведомления выводятся с помощью модальных диалогов JOptionPane.

5. Описание результатов работы программы

Для начала использования необходимо запустить сервер. Если сервер успешно запустился, то он проинформирует пользователя об этом сообщением как показано на рисунке 16.

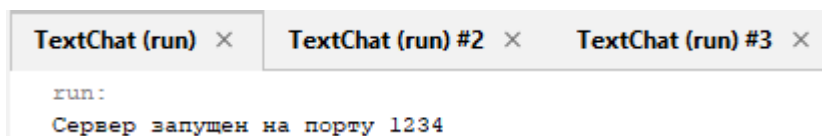


Рисунок 16 – Запуск сервера.

Далее необходимо запустить клиентское приложение. При запуске приложения сначала нужно будет ввести данные о сервере (рисунок 17), к которому необходимо подключиться.

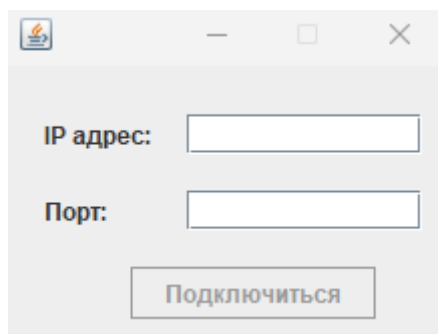


Рисунок 17 – Подключение к серверу.

Если подключение успешно (в противном случае приложение уведомит Вас об ошибке подключения с расшифровкой ошибки), то откроется форма аутентификации и авторизации (рисунок 18). В форме необходимо ввести пару логин-пароль и место, куда будут сохраняться файлы, отправленные другими

пользователями. После этого необходимо выбрать действие – Войти или Зарегистрироваться. Вход предполагает наличие на сервере уже существующего аккаунта, а регистрация – создание нового аккаунта.

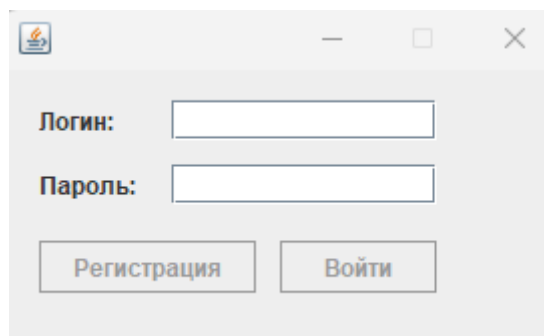
A screenshot of a small application window titled with a flame icon. It contains two text input fields labeled "Логин:" and "Пароль:". Below these fields are two buttons: "Регистрация" and "Войти".

Рисунок 18 – Форма аутентификации и авторизации.

После успешной аутентификации открывается главное окно приложения, как показано на рисунке 19.

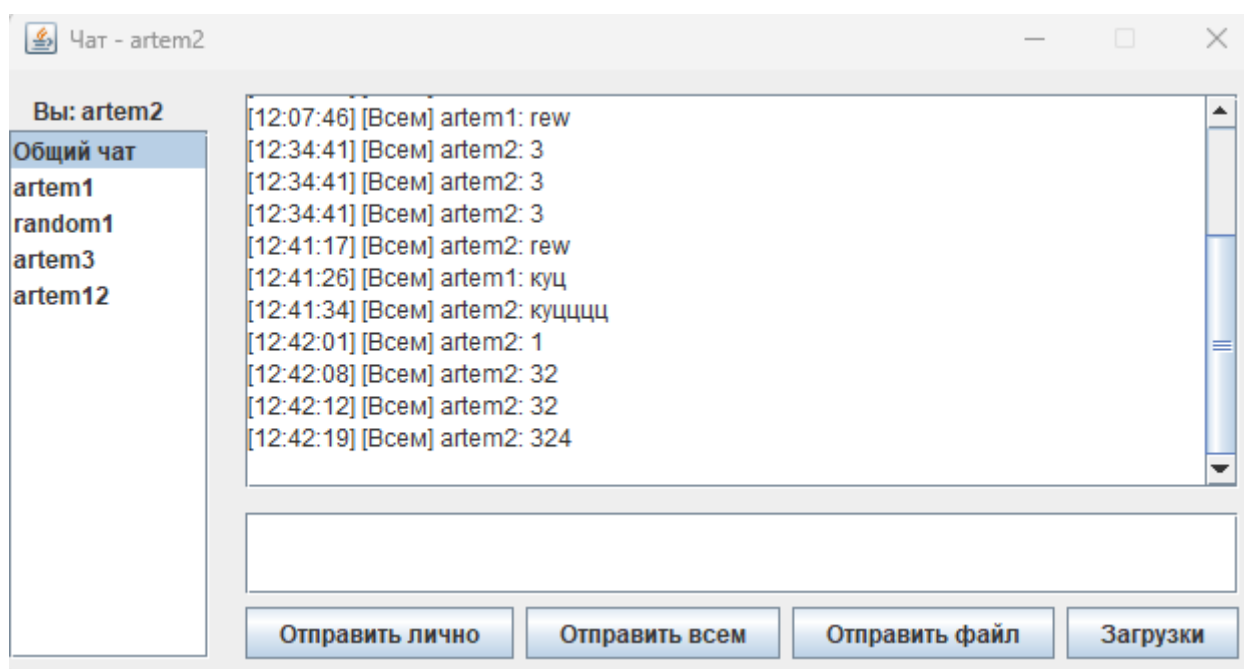
A screenshot of a chat application window titled "Чат - artem2". On the left is a sidebar with a list of users: "Общий чат", "artem1", "random1", "artem3", and "artem12". The main area displays a chat log with timestamps and messages from "artem1" and "artem2". At the bottom, there is a text input field and four buttons: "Отправить лично", "Отправить всем", "Отправить файл", and "Загрузки".

Рисунок 19 – Главное окно приложения.

Для ввода сообщений необходимо написать текст в поле и нажать соответствующую кнопку: “Отправить лично” – сообщение будет отправлено пользователю, выбранному в панели; “Отправить всем” – сообщение будет отправлено всем пользователям.

Для того, чтобы отправить файл необходимо выделить пользователя, которому необходимо отправить файл (как показано на рисунке 20), затем нажать на кнопку “Отправить файл”. Откроется окно выбора файла (рисунок 21).

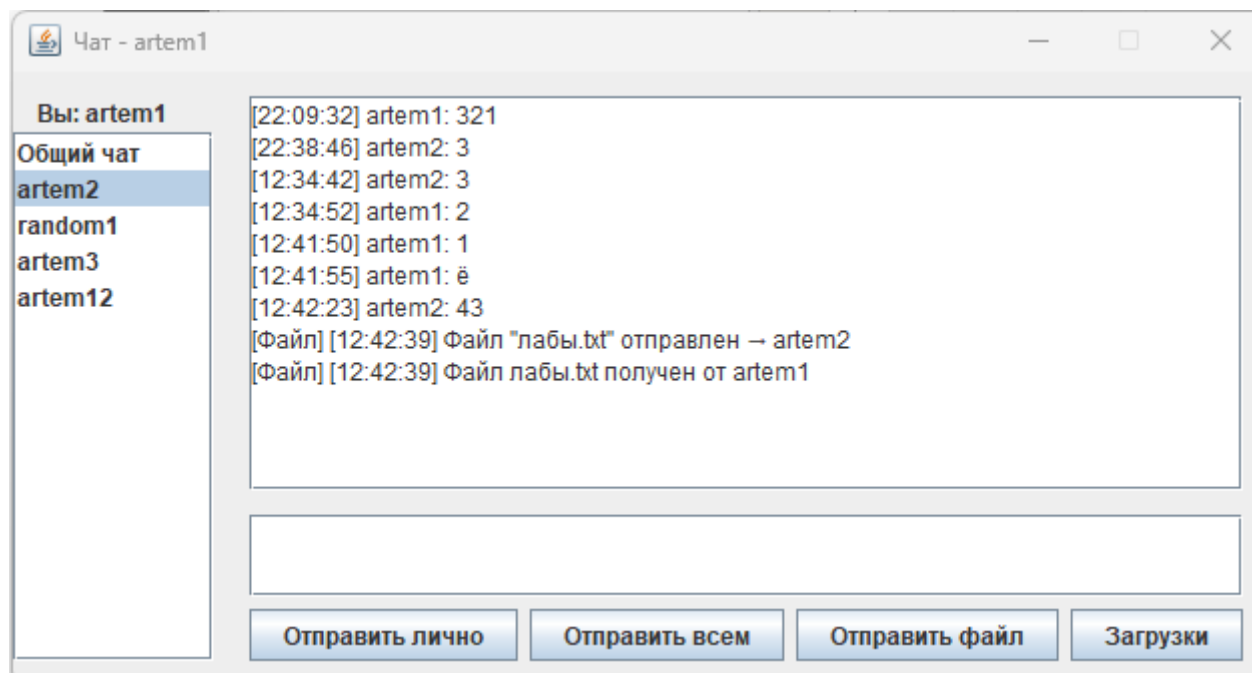


Рисунок 20 – Выбор пользователя.

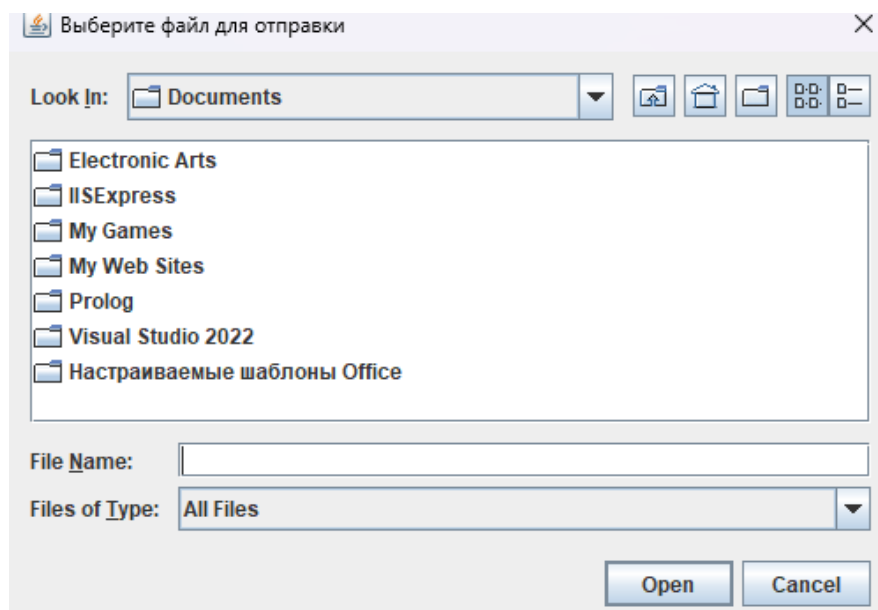


Рисунок 21 – Выбор файла.

После выбора файла необходимо нажать кнопку “Открыть”. Файл будет отправлен выбранному пользователю.

У выбранного пользователя появится сообщение, что ему отправлен файл от другого клиента. По факту получения файла также будет выведено сообщение, что файл получен (рисунок 22).

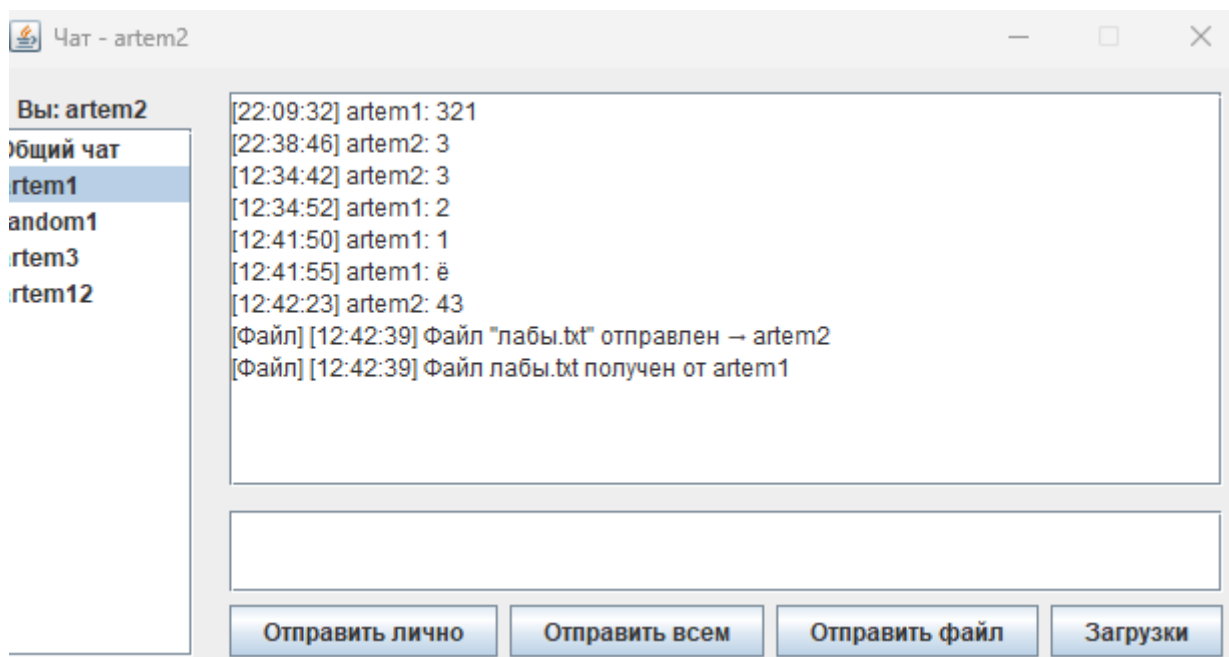


Рисунок 22 – Получение файла пользователем.

Если нужно отправить приватное сообщение, то необходимо выделить пользователя (как показано на рисунке 20), написать текст письма в поле и нажать кнопку “Отправить лично”. Сообщение будет отправлено выделенному пользователю.

Если нужно отправить общее сообщение, то достаточно написать текст письма в поле и нажать кнопку “Отправить всем”. Сообщение будет отправлено всем пользователям.

При корректном выполнении всех шагов система обеспечивает стабильную маршрутизацию сообщений, сохранение истории переписки в локальных логах и успешную передачу файлов без потери данных.

Заключение

В процессе выполнения курсовой работы, посвященной разработке клиент-серверного приложения, были успешно приобретены практические навыки программирования на языке Java с использованием графической библиотеки Java Swing. Работа над проектом позволила углубить понимание принципов создания графического интерфейса пользователя, организации сетевого взаимодействия на основе сокетов TCP/IP, а также многопоточной обработки данных и управления событиями для эффективного контроля элементов интерфейса.

Важным достижением стало модульное структурирование программы, которое значительно упростило процесс разработки и обеспечило удобство дальнейшего расширения системы.

Анализ разработанного мессенджера показывает успешную реализацию базового функционала для обмена сообщениями, включая поддержку различных команд: отправку личных и широковещательных сообщений, получение актуального списка пользователей, управление учетными записями и передачу файлов. Механизмы обработки данных на стороне сервера и клиента демонстрируют стабильное взаимодействие, обеспечивая надежную кроссплатформенную работу приложения.

Для дальнейшего развития мессенджера предусмотрены возможности расширения функциональности. Перспективными направлениями являются внедрение системы восстановления пароля, реализация двухэтапной аутентификации, а также оптимизация протокола обмена данными, что повысит безопасность и производительность использования приложения.

Таким образом, выполнение курсовой работы не только позволило получить ценный практический опыт в области разработки клиент-серверных приложений на платформе Java, но и создало прочную основу для последующего совершенствования и расширения функциональных возможностей приложения, направленных на повышение удобства для конечных пользователей.

Список литературы

1. Шилдт, Герберт Ш57 Java: руководство для начинающих, 7-е изд. : Пер. с англ. - СПб. : ООО "Диалектика", 2019. - 816 с.: ил. - Парал. тит. англ.
2. Хорстманн, К. С. Java. Библиотека профессионала. Том 1 / К. С. Хорстманн. — Санкт-Петербург: Питер, 2020. — 864 с.
3. Васильев, А. Н. Программирование на Java для начинающих / А. Н. Васильев. — Москва : Эксмо, 2017. — 704 с. — (Российский компьютерный бестселлер).
4. Машнин, Т. Многопоточное программирование в Java / Т. Машнин. — Москва : Издательские решения (Ridero), 2024. — 209 с.
5. Дубаков А.А. Сетевое программирование: учебное пособие / А.А. Дубаков – СП: НИУ ИТМО, 2013. – 248 с.

Приложение А. Листинг программы

AuthForm.java:

```
import javax.swing.*;
import java.util.regex.Pattern;
import javax.swing.JOptionPane;

//Форма авторизации
public class AuthForm extends javax.swing.JFrame {
    private ChatForm parentForm; // Ссылка на родительскую форму чата

    //Конструктор формы
    public AuthForm(ChatForm parent) {
        initComponents();
        this.parentForm = parent;

        //Очищаем поля при запуске
        txtLogin.setText("");
        txtPassword.setText("");

        //Отключаем кнопки по умолчанию
        btnLogin.setEnabled(false);
        btnRegister.setEnabled(false);

        //Центрируем окно
        this.setLocationRelativeTo(null);

        //Слушатели для валидации логина и пароля
        txtLogin.getDocument().addDocumentListener(new
javax.swing.event.DocumentListener() {
            public void changedUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
            public void removeUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
            public void insertUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
        });

        txtPassword.getDocument().addDocumentListener(new
javax.swing.event.DocumentListener() {
            public void changedUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
            public void removeUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
            public void insertUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
        });

    }

    //Проверка логина: 3-20 символов, только буквы, цифры, подчёркивания
    private boolean validateLogin(String login) {
        if (login == null || login.trim().isEmpty()) {
            return false;
        }
        if (login.length() < 3 || login.length() > 20) {
            return false;
        }
        // Регулярное выражение: только латиница, цифры, подчёркивания
        return Pattern.matches("[a-zA-Z0-9_]+$", login);
    }

    //Проверка пароля: 6-30 символов
```

```

private boolean validatePassword(String password) {
    if (password == null) {
        return false;
    }
    String trimmedPassword = password.trim();
    if (trimmedPassword.isEmpty()) {
        return false;
    }
    return trimmedPassword.length() >= 6 && trimmedPassword.length() <= 30;
}

//Обновление состояния кнопок при вводе
private void validateInputs() {
    boolean loginValid = validateLogin(txtLogin.getText());
    boolean passValid = validatePassword(new
String(txtPassword.getPassword()));
    btnLogin.setEnabled(loginValid && passValid);
    btnRegister.setEnabled(loginValid && passValid);
}

// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jLabel1 = new javax.swing.JLabel();
    jLabel2 = new javax.swing.JLabel();
    txtLogin = new javax.swing.JTextField();
    btnLogin = new javax.swing.JButton();
    btnRegister = new javax.swing.JButton();
    txtPassword = new javax.swing.JPasswordField();

    setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
    setResizable(false);

    jLabel1.setText("Логин:");

    jLabel2.setText("Пароль:");
    jLabel2.setToolTipText("");

    txtLogin.setText("jTextField1");
    txtLogin.addActionListener(this::txtLoginActionPerformed);

    btnLogin.setText("Войти");
    btnLogin.addActionListener(this::btnLoginActionPerformed);

    btnRegister.setText("Регистрация");
    btnRegister.addActionListener(this::btnRegisterActionPerformed);

    txtPassword.setText("jPasswordField1");
    txtPassword.setEchoChar('*');
    txtPassword.addActionListener(this::txtPasswordActionPerformed);

    javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(

        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
                layout.createSequentialGroup()
                    .addComponent(btnRegister,
                        javax.swing.GroupLayout.DEFAULT_SIZE, 107, Short.MAX_VALUE)
            )
    )
}

```

```

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
    .addComponent(btnLogin,
javax.swing.GroupLayout.PREFERRED_SIZE, 78, javax.swing.GroupLayout.PREFERRED_SIZE))
    .addGroup(javax.swing.GroupLayout.Alignment.LEADING,
layout.createSequentialGroup())

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addComponent(jLabel2)
    .addComponent(jLabel1))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addComponent(txtLogin)
    .addComponent(txtPassword)))
    .addGap(62, 62, 62))
);
layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup())
    .addGap(15, 15, 15)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(jLabel1)
    .addComponent(txtLogin,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(jLabel2)
    .addComponent(txtPassword,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
    .addGap(18, 18, 18)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(btnLogin)
    .addComponent(btnRegister))
    .addContainerGap(30, Short.MAX_VALUE))
);

pack();
setLocationRelativeTo(null);
} // </editor-fold>

private void btnRegisterActionPerformed(java.awt.event.ActionEvent evt) {
String login = txtLogin.getText().trim();
String password = new String(txtPassword.getPassword());

if (!validateLogin(login)) {
    JOptionPane.showMessageDialog(this,
        "Логин: 3-20 символов (буквы, цифры, _)",
        "Ошибка", JOptionPane.ERROR_MESSAGE);
    return;
}

if (!validatePassword(password)) {
    JOptionPane.showMessageDialog(this,
        "Пароль: 6-30 символов",
        "Ошибка", JOptionPane.ERROR_MESSAGE);
    return;
}
}

```



```

    }

    //Блокируем кнопки
    btnLogin.setEnabled(false);
    btnRegister.setEnabled(false);

    //Передаём данные в ChatForm для регистрации
    if (parentForm != null) {
        parentForm.setLoginData(login, password, "register");
    }
}

private void btnLoginActionPerformed(java.awt.event.ActionEvent evt) {
    String login = txtLogin.getText().trim();
    String password = new String(txtPassword.getPassword());

    if (!validateLogin(login)) {
        JOptionPane.showMessageDialog(this,
            "Логин: 3-20 символов (буквы, цифры, _)",
            "Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (!validatePassword(password)) {
        JOptionPane.showMessageDialog(this,
            "Пароль: 6-30 символов",
            "Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    //Блокируем кнопки на время обработки
    btnLogin.setEnabled(false);
    btnRegister.setEnabled(false);

    //Передаём данные в ChatForm для входа
    if (parentForm != null) {
        parentForm.setLoginData(login, password, "login");
    }
}

private void txtLoginActionPerformed(java.awt.event.ActionEvent evt) {
    validateInputs();
}

private void txtPasswordActionPerformed(java.awt.event.ActionEvent evt) {
    validateInputs();
}

//Разблокировать кнопки (вызывается при неудачной авторизации)
public void enableButtons() {
    if (SwingUtilities.isEventDispatchThread()) {
        btnLogin.setEnabled(true);
        btnRegister.setEnabled(true);
    } else {
        SwingUtilities.invokeLater(() -> enableButtons());
    }
}

// Variables declaration - do not modify
private javax.swing.JButton btnLogin;
private javax.swing.JButton btnRegister;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;

```

```

        private javax.swing.JTextField txtLogin;
        private javax.swing.JPasswordField txtPassword;
        // End of variables declaration
    }

```

ChatForm.java:

```

import javax.swing.*.*;
import java.awt.*.*;
import java.io.*.*;
import java.net.*.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Arrays;
import java.util.Base64;
import java.util.List;
import kursach.*;

public class ChatForm extends javax.swing.JFrame {

    //Параметры подключения
    private String serverIP;
    private int serverPort;

    //Данные пользователя
    private String currentUser = null;
    private String currentPassword = null;
    private String selectedUser = "";
    private String localIP = "";

    //Флаги состояния
    private boolean acceptReg = false;
    private boolean shouldContinue = false;

    //Сетевые компоненты
    private Socket tcpClient;
    private Thread clientThread;

    //Таймеры
    private javax.swing.Timer pingTimer;
    private javax.swing.Timer updateTimer;
    private static final int UPLOAD_LIST_INTERVAL = 3000;

    //Директории
    private String logDirectory;
    private String filesDirectory;

    //Ссылка на форму авторизации
    private AuthForm authForm;

    //Конструктор с параметрами подключения
    public ChatForm(String ip, int port) {
        this.serverIP = ip;
        this.serverPort = port;

        //Инициализация директорий
        String userDir = System.getProperty("user.dir");
        logDirectory = userDir + File.separator + "log";
        filesDirectory = userDir + File.separator + "Files";
        new File(logDirectory).mkdirs();
        new File(filesDirectory).mkdirs();
    }

```

```

        initComponents();
        //Очищаем поле ввода
        txtMessageInput.setText("");
        //Сначала форма авторизации
        showAuthForm();
    }

    private void showAuthForm() {
        authForm = new AuthForm(this);
        authForm.setVisible(true);
    }

    //Установка данных логина/пароля из AuthForm
    public void setLoginData(String login, String password, String action) {
        this.currentUser = login;
        this.currentPassword = password;
    }

    try {
        connectToServer();
        startTimers();

        //Задержка, чтобы получить localIP
        Thread.sleep(500);
        getUserLocalIPRequest();

        if ("login".equals(action)) {
            loginUser(password);
        } else {
            registerUser(login, password);
        }
    } catch (Exception ex) {

        ex.printStackTrace();
        JOptionPane.showMessageDialog(this,
            "Ошибка подключения: " + ex.getMessage(), "Ошибка",
            JOptionPane.ERROR_MESSAGE);
        enableAuthFormButtons();
    }
}

    public void closeAuthForm() {
        if (authForm != null) {
            authForm.dispose();
        }
    }

    //Разблокировать кнопки в AuthForm
    public void enableAuthFormButtons() {
        if (authForm != null) {
            authForm.enableButtons();
        }
    }

    //Подключение к серверу
    private void connectToServer() throws IOException {
        tcpClient = new Socket();
        tcpClient.connect(new InetSocketAddress(serverIP, serverPort), 5000);
        shouldContinue = true;

        //Запуск потока для чтения сообщений
        clientThread = new Thread(this::listenForMessages);
        clientThread.setDaemon(true);
    }

```

```

        clientThread.start();
    }

    //Отправка сообщения на сервер
    private void sendMessage(CommandType commandType, String sender, String
receiver, String content) {
        if (tcpClient == null || tcpClient.isClosed()) return;

        try {
            Message msg = new Message(commandType, sender, receiver, content);
            String data = msg.serialize();

            if (data.length() > 65500) {
                SwingUtilities.invokeLater(() ->
JOptionPane.showMessageDialog(this, "Сообщение слишком большое"));
                return;
            }

            OutputStream out = tcpClient.getOutputStream();

            out.write((data + "\n").getBytes(StandardCharsets.UTF_8));
            out.flush();

            } catch (IOException ex) {
                System.err.println("Не удалось отправить сообщение: " +
ex.getMessage());
            }
        }

        private void loginUser(String password) {
            sendMessage(CommandType.Login, localIP, "", currentUser + "," +
password);
        }

        private void registerUser(String login, String password) {
            sendMessage(CommandType.Register, localIP, "", login + "," + password);
        }

        private void getUserLocalIPRequest() {
            sendMessage(CommandType.GetUserLocalIP, "", "", "");
        }

        private void listenForMessages() {

            try {
                InputStream in = tcpClient.getInputStream();
                BufferedReader reader = new BufferedReader(
                    new InputStreamReader(in, StandardCharsets.UTF_8));

                String line;
                while (shouldContinue && (line = reader.readLine()) != null) {

                    try {
                        Message msg = Message.deserialize(line);
                        processCommand(msg);
                    } catch (Exception ex) {
                        System.err.println("Ошибка десериализации: " + ex.getMessage());
                    }
                }

            } catch (IOException ex) {
                if (shouldContinue) {
                    System.err.println("Соединение с сервером потеряно: " +

```

```

ex.getMessage());
        SwingUtilities.invokeLater(() -> {
            JOptionPane.showMessageDialog(this,
                "Соединение с сервером потеряно.",
                "Ошибка", JOptionPane.ERROR_MESSAGE);
        });
    }
}

private void processCommand(Message msg) {
    SwingUtilities.invokeLater(() -> {
        try {
            switch (msg.getCommandType()) {
                case UserListUpdate:
                    updateUserList(msg.getContent());
                    break;
                case GetUserLocalIP:
                    localIP = msg.getContent();
                    System.out.println("Мой локальный IP: " + localIP);
                    break;
                case SendPrivateMessage:
                    handlePrivateMessage(msg);
                    break;
                case SendBroadcastMessage:
                    handleBroadcastMessage(msg);
                    break;
                case SendFile:
                    createFile(msg.getSender(), msg.getContent());
                    break;
                case Register:
                    checkRegister(msg.getContent());
                    break;
                case Login:
                    checkLogin(msg.getContent());
                    break;
                case Ping:
                    break;
            }
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    });
}

private void handlePrivateMessage(Message msg) {
    String dialogFile = getDialogFileName(msg.getSender(), currentUser);
    String messageToRich = "[" +
        LocalTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss")) + "] " +
        msg.getSender() + ": " + msg.getContent();
    saveMessageToLog(dialogFile, messageToRich);

    //Показываем сообщение, только если открыт диалог с этим пользователем
    if (selectedUser != null && selectedUser.equals(msg.getSender())) {
        txtChatLog.append(messageToRich + "\n");
    }
}

private void handleBroadcastMessage(Message msg) {
    String messageToRich = "[" +
        LocalTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss")) + "] [Всем] " +
        msg.getSender() + ": " + msg.getContent();
    txtChatLog.append(messageToRich + "\n");
    saveMessageToLog("ALL", messageToRich);
}

```

```

private void checkLogin(String content) {
    String response = content.trim();
    acceptReg = false;

    if ("True".equals(response)) {
        acceptReg = true;
        lbCurrentUser.setText("Вы: " + currentUser);
        setTitle("Чат - " + currentUser);

        SwingUtilities.invokeLater(() -> {
            this.setLocationRelativeTo(null);
            this.setVisible(true);
            thisToFront();
        });
        closeAuthForm();
        sendMessage(CommandType.UserListUpdate, "", "", "");
    } else {

        JOptionPane.showMessageDialog(this,
            "Неверный логин/пароль или пользователь в сети",
            "Ошибка входа", JOptionPane.ERROR_MESSAGE);
        enableAuthFormButtons();
    }
}

private void checkRegister(String content) {

    if ("False".equals(content.trim())) {
        acceptReg = false;

        JOptionPane.showMessageDialog(this, "Такой логин уже существует");
        enableAuthFormButtons();
    } else if ("True".equals(content.trim())) {
        acceptReg = true;

        //Небольшая задержка перед авто-входом
        new Thread(() -> {
            try {
                Thread.sleep(1000);

                loginUser(currentPassword);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }).start();
    }
}

private void updateUserList(String userList) {
    if (acceptReg && userList != null && !userList.isEmpty()) {
        //Сохраняем текущий выбранный элемент
        String previouslySelected = lstUsers.getSelectedValue();
        String[] userArray = userList.split(",");
        DefaultListModel<String> model = new DefaultListModel<>();

        //Элемент для общего чата в начало списка
        model.addElement("Общий чат");

        //Используем Stream API
        Arrays.stream(userArray)
            .filter(user -> !user.isEmpty() && !user.equals(currentUser))
            .forEach(model::addElement);
    }
}

```

```

lstUsers.setModel(model);

//Восстанавливаем выбор, если пользователь всё ещё в списке
if (previouslySelected != null && model.contains(previouslySelected)) {
    lstUsers.setSelectedValue(previouslySelected, true);
    selectedUser = previouslySelected;
} else {
    // Если ничего не выбрано или пользователь вышел – выбираем общий
чат
    lstUsers.setSelectedIndex(0);
    selectedUser = null;
}
}

private String getDialogFileName(String user1, String user2) {
    // Проверка на null
    String u1 = (user1 != null) ? user1 : "unknown";
    String u2 = (user2 != null) ? user2 : "unknown";

    // Сравниваем и возвращаем в алфавитном порядке
    if (u1.compareTo(u2) < 0) {
        return u1 + "_" + u2;
    } else {
        return u2 + "_" + u1;
    }
}

private void saveMessageToLog(String dialogName, String message) {
    String filePath = logDirectory + File.separator + dialogName +
"_chatlog.txt";
    try (PrintWriter writer = new PrintWriter(new FileWriter(filePath,
true))) {
        writer.println(message);
    } catch (IOException ex) {
        System.err.println("Ошибка записи в лог: " + ex.getMessage());
    }
}

private void createFile(String sender, String info) {
    try {
        String[] parts = info.split("\\|", 2);
        if (parts.length < 2) {
            JOptionPane.showMessageDialog(this,
                "Ошибка: неверный формат файла",
                "Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }

        String fileName = parts[0].trim();
        String fileDataBase64 = parts[1].trim();

        // Создаём файл в папке Files
        String destPath = filesDirectory + File.separator + fileName;
        byte[] fileData = Base64.getDecoder().decode(fileDataBase64);
        Files.write(Paths.get(destPath), fileData);

        String message = "[" + LocalTime.now().format(
            DateTimeFormatter.ofPattern("HH:mm:ss")) +
            "]" + "Файл " + fileName + " получен от " + sender;

        txtChatLog.append(message + "\n");

        //Проверка на null

```



```

        if (currentUser != null && sender != null) {
            String dialogFile = getDialogFileName(sender, currentUser);
            saveMessageToLog(dialogFile, "[Файл] " + message);
        }

        JOptionPane.showMessageDialog(this,
            "Файл получен: " + fileName + "\nСохранён в: " + destPath,
            "Файл получен", JOptionPane.INFORMATION_MESSAGE);

    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this,
            "Ошибка при создании файла: " + ex.getMessage(),
            "Ошибка", JOptionPane.ERROR_MESSAGE);
    }
}

private void startTimers() {
    pingTimer = new javax.swing.Timer(5000, e ->
sendMessage(CommandType.Ping, "", "", ""));
    pingTimer.start();

    updateTimer = new javax.swing.Timer(UPLOAD_LIST_INTERVAL, e ->
sendMessage(CommandType.UserListUpdate, "", "", ""));
    updateTimer.start();
}

private void transferFile(String receiver, String filePath) {
    try {
        // Проверки
        if (receiver == null || receiver.isEmpty()) {
            SwingUtilities.invokeLater(() ->
                JOptionPane.showMessageDialog(this,
                    "Ошибка: не выбран получатель!",
                    "Ошибка", JOptionPane.ERROR_MESSAGE));
            return;
        }

        if (currentUser == null) {
            SwingUtilities.invokeLater(() ->
                JOptionPane.showMessageDialog(this,
                    "Ошибка: вы не авторизованы!",
                    "Ошибка", JOptionPane.ERROR_MESSAGE));
            return;
        }

        // Читаем файл
        byte[] fileData = Files.readAllBytes(Paths.get(filePath));
        String fileName = new File(filePath).getName();

        // Кодировем в Base64
        String fileInfo = fileName + "|" +
Base64.getEncoder().encodeToString(fileData);

        // Проверяем размер (макс ~65KB в Base64)
        if (fileInfo.length() > 65000) {
            SwingUtilities.invokeLater(() ->
                JOptionPane.showMessageDialog(this,
                    "Файл слишком большой! Максимальный размер ~48KB",
                    "Ошибка", JOptionPane.ERROR_MESSAGE));
            return;
        }
    }
}

```

```

//Отправляем файл

sendMessage(CommandType.SendFile, currentUser, receiver, fileInfo);

//Показываем сообщение в своём чате
String message = "[" + LocalTime.now().format(
    DateTimeFormatter.ofPattern("HH:mm:ss")) +
    "]" + "Файл \"" + fileName + "\" отправлен → " + receiver;

SwingUtilities.invokeLater(() -> {
    txtChatLog.append(message + "\n");
});

//Сохраняем в лог
String dialogFile = getDialogFileName(currentUser, receiver);
saveMessageToLog(dialogFile, "[Файл] " + message);

//Уведомление
SwingUtilities.invokeLater(() ->
    JOptionPane.showMessageDialog(this,
        "📎 ФАЙЛ ОТПРАВЛЕН!\n\nИмя: " + fileName +
        "\nКому: " + receiver +
        "\nРазмер: " + fileData.length + " байт",
        "Файл отправлен", JOptionPane.INFORMATION_MESSAGE));

} catch (Exception ex) {

    ex.printStackTrace();
    SwingUtilities.invokeLater(() ->
        JOptionPane.showMessageDialog(this,
            "Ошибка при отправке файла:\n" + ex.getMessage(),
            "Ошибка", JOptionPane.ERROR_MESSAGE));
}

}

// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jScrollPane2 = new javax.swing.JScrollPane();
    lstUsers = new javax.swing.JList<>();
    jScrollPane1 = new javax.swing.JScrollPane();
    txtChatLog = new javax.swing.JTextArea();
    txtMessageInput = new javax.swing.JTextField();
    btnSendPrivate = new javax.swing.JButton();
    btnSendAll = new javax.swing.JButton();
    btnSendFile = new javax.swing.JButton();
    btnOpenFolder = new javax.swing.JButton();
    lbCurrentUser = new javax.swing.JLabel();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
    setResizable(false);
    addWindowListener(new java.awt.event.WindowAdapter() {
        public void windowClosing(java.awt.event.WindowEvent evt) {
            formWindowClosing(evt);
        }
    });

    lstUsers.addListSelectionListener(this::lstUsersValueChanged);
    jScrollPane2.setViewportViewView(lstUsers);

    txtChatLog.setEditable(false);
    txtChatLog.setColumns(20);
    txtChatLog.setRows(5);
    jScrollPane1.setViewportViewView(txtChatLog);

```

```

txtMessageInput.setText("jTextField1");

btnSendPrivate.setText("Отправить лично");
btnSendPrivate.addActionListener(this::btnSendPrivateActionPerformed);

btnSendAll.setText("Отправить всем");
btnSendAll.addActionListener(this::btnSendAllActionPerformed);

btnSendFile.setText("Отправить файл");
btnSendFile.addActionListener(this::btnSendFileActionPerformed);

btnOpenFolder.setText("Загрузки");
btnOpenFolder.addActionListener(this::btnOpenFolderActionPerformed);

lbCurrentUser.setText("Вы:");

javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addComponent(jScrollPane2,
javax.swing.GroupLayout.PREFERRED_SIZE, 100, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGroup(layout.createSequentialGroup()
            .addGap(18, 18, 18)
            .addComponent(lbCurrentUser))
        .addGap(18, 18, 18)
        .addGroup(layout.createSequentialGroup()
            .addComponent(btnSendPrivate)
            .addGap(18, 18, 18)
            .addComponent(btnSendAll)
            .addGap(18, 18, 18)
            .addComponent(btnSendFile)
            .addGap(18, 18, 18)
            .addComponent(btnOpenFolder)
            .addComponent(jScrollPane1)
            .addComponent(txtMessageInput)
        )
    )
);
layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addComponent(lbCurrentUser)
        .addGap(2, 2, 2)
        .addComponent(jScrollPane2)
        .addGap(2, 2, 2)
        .addComponent(jScrollPane1)
        .addGap(2, 2, 2)
        .addComponent(txtMessageInput)
    )
);

```

```

        .addComponent(txtMessageInput,
javax.swing.GroupLayout.PREFERRED_SIZE, 41, javax.swing.GroupLayout.PREFERRED_SIZE)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(btnSendPrivate)
            .addComponent(btnSendAll)
            .addComponent(btnSendFile)
            .addComponent(btnOpenFolder))
            .addGap(0, 0, Short.MAX_VALUE))
        .addContainerGap()
    );

    pack();
    setLocationRelativeTo(null);
} // </editor-fold>

private void btnSendPrivateActionPerformed(java.awt.event.ActionEvent evt) {
    String message = txtMessageInput.getText().trim();
    if (message.isEmpty()) return;

    //Получаем выбранного пользователя
    String selected = lstUsers.getSelectedValue();

    //Проверяем, не выбран ли "Общий чат"
    if (selected == null || selected.isEmpty() || selected.equals("Общий чат"))
    {
        JOptionPane.showMessageDialog(this,
            "Выберите пользователя из списка для личного сообщения!");
        return;
    }

    selectedUser = selected;
    sendMessage(CommandType.SendPrivateMessage, currentUser, selectedUser,
message);
    txtMessageInput.setText("");
}

private void btnSendAllActionPerformed(java.awt.event.ActionEvent evt) {
    String message = txtMessageInput.getText().trim();
    if (message.isEmpty()) return;

    sendMessage(CommandType.SendBroadcastMessage, currentUser, "", message);
    txtMessageInput.setText("");
}

private void btnSendFileActionPerformed(java.awt.event.ActionEvent evt) {
    //Получаем выбранного пользователя
    String selected = lstUsers.getSelectedValue();

    if (selected == null || selected.isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "Выберите пользователя из списка!",
            "Ошибка", JOptionPane.WARNING_MESSAGE);
        return;
    }

    if (selected == null || selected.isEmpty() || selected.equals("Общий чат"))
    {
        JOptionPane.showMessageDialog(this,
            "Выберите конкретного пользователя для отправки файла!",
            "Ошибка", JOptionPane.WARNING_MESSAGE);
        return;
    }
}

```

```

    }

    //Сохраняем выбранного пользователя
    selectedUser = selected;

    //Открываем диалог выбора файла
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setDialogTitle("Выберите файл для отправки");

    if (fileChooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
        File selectedFile = fileChooser.getSelectedFile();

        //Запускаем отправку в отдельном потоке
        String receiver = selectedUser; // Сохраняем в final переменную
        Thread fileThread = new Thread(() ->
            transferFile(receiver, selectedFile.getAbsolutePath()));
        fileThread.setDaemon(true);
        fileThread.start();
    }
}

private void btnOpenFolderActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        new File(filesDirectory).mkdirs();
        Desktop.getDesktop().open(new File(filesDirectory));
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Ошибка при открытии папки: " +
ex.getMessage());
    }
}

private void lstUsersValueChanged(javax.swing.event.ListSelectionEvent evt)
{
    String selected = lstUsers.getSelectedValue();

    //Проверяем, выбран ли "Общий чат"
    if (selected != null && selected.equals("Общий чат")) {
        selectedUser = null; // Сбрасываем выбранного пользователя
        txtChatLog.setText("");

        //Показываем общий лог
        String filePath = logDirectory + File.separator + "ALL_chatlog.txt";
        try {
            if (new File(filePath).exists()) {
                String chatLog = new String(java.nio.file.Files.readAllBytes(
                    java.nio.file.Paths.get(filePath)), StandardCharsets.UTF_8);
                txtChatLog.setText(chatLog);
            }
        } catch (IOException ex) {
            System.err.println("Ошибка чтения общего лога: " + ex.getMessage());
        }
    } else if (selected != null) {
        //Выбран конкретный пользователь
        selectedUser = selected;
        txtChatLog.setText("");

        String dialogFile = getDialogFileName(currentUser, selectedUser);
        String filePath = logDirectory + File.separator + dialogFile +
"_chatlog.txt";
        try {
            if (new File(filePath).exists()) {
                String chatLog = new String(java.nio.file.Files.readAllBytes(
                    java.nio.file.Paths.get(filePath)), StandardCharsets.UTF_8);

```

```

        txtChatLog.setText(chatLog);
    }
    } catch (IOException ex) {
        JOptionPane.showMessageDialog(this, "Ошибка при чтении файла: " +
ex.getMessage());
    }
}
}

private void formWindowClosing(java.awt.event.WindowEvent evt) {
    shouldContinue = false;
    if (pingTimer != null) pingTimer.stop();
    if (updateTimer != null) updateTimer.stop();
    if (tcpClient != null) {
        try { tcpClient.close(); } catch (IOException ignored) {}
    }
}

// Variables declaration - do not modify
private javax.swing.JButton btnOpenFolder;
private javax.swing.JButton btnSendAll;
private javax.swing.JButton btnSendFile;
private javax.swing.JButton btnSendPrivate;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JScrollPane jScrollPane2;
private javax.swing.JLabel lbCurrentUser;
private javax.swing.JList<String> lstUsers;
private javax.swing.JTextArea txtChatLog;
private javax.swing.JTextField txtMessageInput;
// End of variables declaration

```

ConnectionForm1.java:

```

import javax.swing.JOptionPane;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.io.IOException;

//Форма подключения к серверу
public class ConnectionForm1 extends javax.swing.JFrame {

    //Поля для хранения данных подключения
    private String serverIP;
    private int serverPort;

    //Конструктор формы
    public ConnectionForm1() {
        initComponents();
        txtIP.setText("");
        txtPort.setText("");
        btnConnect.setEnabled(false); // Кнопка неактивна до валидного ввода

        //Окно по центру экрана
        this.setLocationRelativeTo(null);

        //Слушатели для валидации ввода
        txtIP.getDocument().addDocumentListener(new
javax.swing.event.DocumentListener() {
            public void changedUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
            public void removeUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
            public void insertUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
        });
    }
}

```

```

        txtPort.getDocument().addDocumentListener(new
javax.swing.event.DocumentListener() {
    public void changedUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
    public void removeUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
    public void insertUpdate(javax.swing.event.DocumentEvent e) {
validateInputs(); }
});
}

//Обновление состояния кнопки при вводе
private void validateInputs() {
boolean ipValid = validateIPAddress(txtIP.getText());
boolean portValid = validatePort(txtPort.getText());
btnConnect.setEnabled(ipValid && portValid);
}

//Геттеры для получения данных после подключения
public String getServerIP() {
    return serverIP;
}
public int getServerPort() {
    return serverPort;
}

//Метод проверки IP адреса
private boolean validateIPAddress(String ip) {
    if (ip == null || ip.trim().isEmpty()) {
        return false;
    }

    if (ip.equals("localhost")) {
        return true;
    }

    //Регулярное выражение для IPv4
    String pattern = "^((25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\\.){3}" +
        "(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$";
    return ip.matches(pattern);
}

//Метод проверки порта от 1 до 65535
private boolean validatePort(String port) {
    if (port == null || port.trim().isEmpty()) {
        return false;
    }

    try {
        int portNumber = Integer.parseInt(port);
        return portNumber >= 1 && portNumber <= 65535;
    } catch (NumberFormatException e) {
        return false;
    }
}

// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jLabel1 = new javax.swing.JLabel();
    jLabel2 = new javax.swing.JLabel();
    txtIP = new javax.swing.JTextField();
    txtPort = new javax.swing.JTextField();
    btnConnect = new javax.swing.JButton();

```

```

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
setResizable(false);

jLabel1.setText("IP адрес:");

jLabel2.setText("Порт:");

txtIP.setText("setText");

txtPort.setText("setText");

btnConnect.setText("Подключиться");
btnConnect.addActionListener(this::btnConnectActionPerformed);

javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addGap(20, 20, 20)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(jLabel1)
            .addComponent(jLabel2))
        .addGap(18, 18, 18)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(txtPort)
            .addComponent(txtIP))
            .addGap(63, 63, 63)
            .addComponent(btnConnect)
            .addGap(35, Short.MAX_VALUE))
    );
layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addGap(24, 24, 24)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(jLabel1)
            .addComponent(txtIP, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))
            .addGap(18, 18, 18)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(jLabel2)
            .addComponent(txtPort,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
            .addGap(18, 18, 18)
            .addComponent(btnConnect)
            .addGap(Short.MAX_VALUE))
    );

pack();
setLocationRelativeTo(null);
} // </editor-fold>

```



```

private void btnConnectActionPerformed(java.awt.event.ActionEvent evt) {
    String ip = txtIP.getText().trim();
    String portStr = txtPort.getText().trim();

    //Проверяем IP адрес
    if (!validateIPAddress(ip)) {
        JOptionPane.showMessageDialog(this,
            "Неверный формат IP адреса", "Ошибка",
            JOptionPane.ERROR_MESSAGE);
        return;
    }

    //Проверяем порт
    if (!validatePort(portStr)) {
        JOptionPane.showMessageDialog(this,
            "Порт должен быть числом от 1 до 65535", "Ошибка",
            JOptionPane.ERROR_MESSAGE);
        return;
    }

    //Сохраняем данные
    this.serverIP = ip;
    this.serverPort = Integer.parseInt(portStr);

    if (!checkServerAvailable(serverIP, serverPort)) {
        JOptionPane.showMessageDialog(this,
            "Сервер недоступен!\nПроверьте IP, порт и запущен ли сервер.",
            "Ошибка подключения",
            JOptionPane.ERROR_MESSAGE);
        return;
    }

    //Открываем главное окно чата
    ChatForm chatForm = new ChatForm(serverIP, serverPort);

    //Закрываем текущую форму
    this.dispose();
}

private boolean checkServerAvailable(String ip, int port) {
    try (Socket testSocket = new Socket()) {
        testSocket.connect(new InetSocketAddress(ip, port), 3000); // Таймаут 3
секунды
        return true;
    }
    catch (IOException e) {
        return false;
    }
}

public static void main(String args[]) {
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (Exception ex) {
        ex.printStackTrace();
    }
    java.awt.EventQueue.invokeLater(() -> new
ConnectionForm1().setVisible(true));
}

```

```

        // Variables declaration - do not modify
        private javax.swing.JButton btnConnect;
        private javax.swing.JLabel jLabel1;
        private javax.swing.JLabel jLabel2;
        private javax.swing.JTextField txtIP;
        private javax.swing.JTextField txtPort;
        // End of variables declaration
    }

```

Main.java:

```

import javax.swing.SwingUtilities;
import java.io.PrintStream;
import java.io.UnsupportedEncodingException;

public class Main {
    public static void main(String[] args) {
        try {
            //Кодировка для консоли
            System.setOut(new PrintStream(System.out, true, "UTF-8"));
            System.setErr(new PrintStream(System.err, true, "UTF-8"));
        } catch (UnsupportedEncodingException e) {
            System.err.println("Ошибка установки кодировки: " + e.getMessage());
        }

        SwingUtilities.invokeLater(() -> {
            ConnectionForm1 connectForm = new ConnectionForm1();
            connectForm.setVisible(true);
        });
    }
}

```

ClientHandler.java:

```

package server;

import kursach.*;
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.util.List;

//Обработчик подключенного клиента (выполняется в отдельном потоке)
public class ClientHandler implements Runnable {
    private final Socket clientSocket;
    private final String clientIP;
    private String login = null;
    private volatile boolean connected = true;
    private final UserManager userManager;
    private final List<ClientHandler> clients;

    public ClientHandler(Socket socket, String ip, UserManager userManager,
        List<ClientHandler> clients) {
        this.clientSocket = socket;
        this.clientIP = ip;
    }
}

```

```

        this.userManager = userManager;
        this.clients = clients;
    }

    @Override
    public void run() {
        try (BufferedReader reader = new BufferedReader(
            new InputStreamReader(clientSocket.getInputStream(),
StandardCharsets.UTF_8))) {

            String line;
            while (connected && (line = reader.readLine()) != null) {
                try {
                    Message msg = Message.deserialize(line);
                    processMessage(msg);
                } catch (ChatException ex) {
                    System.err.println("Ошибка обработки сообщения от " +
clientIP + ": " + ex.getMessage());
                }
            }
        } catch (IOException e) {
            if (connected) {
                System.err.println("Ошибка чтения от " + clientIP + ": " +
e.getMessage());
            }
        } finally {
            disconnect();
        }
    }

    //Отправка сообщения клиенту (с символом новой строки для readLine())
    public boolean sendMessage(Message msg) {
        if (!connected || clientSocket.isClosed()) {
            return false;
        }

        try {
            String data = msg.serialize() + "\n";
            OutputStream out = clientSocket.getOutputStream();
            out.write(data.getBytes(StandardCharsets.UTF_8));
            out.flush();
            return true;
        } catch (IOException e) {
            System.err.println("Ошибка отправки клиенту " + clientIP + ": " +
e.getMessage());
            disconnect();
            return false;
        }
    }

```

```

    }
}

//Отключение клиента
public void disconnect() {
    if (connected) {
        connected = false;
        try {
            clientSocket.close();
        } catch (IOException ignored) {}

        System.out.println("Клиент отключён: " + clientIP + " (логин: " +
login + ")");

        synchronized (clients) {
            clients.remove(this);
        }

        //Рассылаем обновлённый список пользователей всем
        broadcastUserList();
    }
}

//Обработка сообщения по типу команды
private void processMessage(Message msg) {
    switch (msg.getCommandType()) {
        case Ping:
            handlePing();
            break;
        case UserListUpdate:
            handleUserListUpdate();
            break;
        case GetUserLocalIP:
            handleGetLocalIP();
            break;
        case Login:
            handleLogin(msg);
            break;
        case Register:
            handleRegister(msg);
            break;
        case SendPrivateMessage:
            handlePrivateMessage(msg);
            break;
        case SendBroadcastMessage:
            handleBroadcastMessage(msg);

```

```

        break;
    case SendFile:
        handleFileTransfer(msg);
        break;
    }
}

//Обработка Ping - просто отвечаем таким же Ping
private void handlePing() {
    sendMessage(new Message(CommandType.Ping, "", "", ""));
}

//Отправка списка пользователей текущему клиенту
private void handleUserListUpdate() {
    List<String> allUsers = userManager.getAllLogins();
    String userList = String.join(",", allUsers);
    sendMessage(new Message(CommandType.UserListUpdate, "", "", userList));
}

//Отправка клиенту его локального IP
private void handleGetLocalIP() {
    sendMessage(new Message(CommandType.GetUserLocalIP, "", "", clientIP));
}

//Обработка входа в систему
private void handleLogin(Message msg) {
    String[] creds = msg.getContent().split(",", 2);
    if (creds.length != 2) {
        sendLoginResponse(false);
        return;
    }
    String loginAttempt = creds[0];
    String passwordAttempt = creds[1];

    //Проверяем корректность логина и пароля
    if (!userManager.validateLogin(loginAttempt, passwordAttempt)) {
        sendLoginResponse(false);
        System.out.println("Неудачная попытка входа: " + loginAttempt + " с
IP: " + clientIP);
        return;
    }

    //Проверяем не занят ли логин другим активным клиентом
    boolean isAlreadyLoggedIn = false;
    synchronized (clients) {
        for (ClientHandler client : clients) {

```

```

        //Исключаем текущего клиента, проверяем активность и совпадение
логина
        if (client != this && client.connected && client.login != null
&& client.login.equals(loginAttempt)) {
            isAlreadyLoggedIn = true;
            break;
        }
    }

    if (isAlreadyLoggedIn) {
        sendLoginResponse(false);
        System.out.println("Вход отклонён: пользователь '" + loginAttempt +
"'" уже в сети с другого устройства.");
        return;
    }

    //Успешная авторизация
    this.login = loginAttempt;
    sendLoginResponse(true);
    System.out.println("Пользователь '" + login + "'" вошёл с IP: " +
clientIP);
    broadcastUserList(); // Обновляем списки всех пользователей
}

//Обработка регистрации нового пользователя
private void handleRegister(Message msg) {
    String[] creds = msg.getContent().split(",", 2);

    if (creds.length != 2) {
        sendRegisterResponse(false);
        return;
    }

    String newLogin = creds[0];
    String newPassword = creds[1];

    if (userManager.registerUser(newLogin, newPassword)) {
        sendRegisterResponse(true);
        System.out.println("Зарегистрирован новый пользователь: " +
newLogin);
    } else {
        sendRegisterResponse(false);
        System.out.println("Ошибка регистрации (логин уже существует): " +
newLogin);
    }
}
}

```

```

//Пересылка личного сообщения указанному получателю
private void handlePrivateMessage(Message msg) {
    boolean delivered = false;

    synchronized (clients) {
        for (ClientHandler client : clients) {
            if (client.login != null &&
                client.login.equals(msg.getReceiver()) &&
                client.connected) {

                Message forward = new Message(
                    CommandType.SendPrivateMessage,
                    msg.getSender(),
                    msg.getReceiver(),
                    msg.getContent()
                );
                client.sendMessage(forward);
                delivered = true;
                break;
            }
        }
    }

    if (!delivered) {
        System.out.println("Не удалось доставить личное сообщение
пользователю: " + msg.getReceiver());
    }
}

//Рассылка широковещательного сообщения всем (кроме отправителя)
private void handleBroadcastMessage(Message msg) {
    synchronized (clients) {
        for (ClientHandler client : clients) {
            if (client != this && client.connected) {
                Message forward = new Message(
                    CommandType.SendBroadcastMessage,
                    msg.getSender(),
                    "",
                    msg.getContent()
                );
                client.sendMessage(forward);
            }
        }
    }
}

```

```

//Пересылка файла указанному получателю
private void handleFileTransfer(Message msg) {

    boolean delivered = false;

    synchronized (clients) {
        for (ClientHandler client : clients) {

            if (client.login != null &&
                client.login.equals(msg.getReceiver()) &&
                client.connected) {

                Message forward = new Message(
                    CommandType.SendFile,
                    msg.getSender(),
                    msg.getReceiver(),
                    msg.getContent()
                );

                boolean sent = client.sendMessage(forward);

                delivered = true;
                break;
            }
        }
    }

    if (!delivered) {

        synchronized (clients) {
            for (ClientHandler c : clients) {
            }
        }
    }
}

//Отправка ответа на попытку входа
private void sendLoginResponse(boolean success) {
    sendMessage(new Message(CommandType.Login, "", "", success ? "True" :
"False"));
}

//Отправка ответа на попытку регистрации

```



```

        private void sendRegisterResponse(boolean success) {
            sendMessage(new Message(CommandType.Register, "", "", success ? "True" :
"False"));
        }

        //Рассылка обновлённого списка пользователей всем подключенным клиентам
        private void broadcastUserList() {
            List<String> allUsers = userManager.getAllLogins();
            String userList = String.join(",", allUsers);

            synchronized (clients) {
                for (ClientHandler client : clients) {
                    if (client.connected) {
                        client.sendMessage(new Message(CommandType.UserListUpdate,
"", "", userList));
                    }
                }
            }
        }

        //Геттеры
        public String getLogin() {
            return login;
        }

        public boolean isConnected() {
            return connected;
        }

        public String getClientIP() {
            return clientIP;
        }
    }
}

```

Server.java:

```

package server;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.ArrayList;
import java.util.List;
import java.io.UnsupportedEncodingException;
import java.io.PrintStream;

//Основной класс сервера чата
public class Server {
    private static final int PORT = 1234;
    private final ServerSocket serverSocket;
    private final UserManager userManager;
    private final List<ClientHandler> clients = new ArrayList<>();
}

```

```

private volatile boolean running = true;

public Server(int port) throws IOException {
    this.serverSocket = new ServerSocket(port);
    this.userManager = new UserManager();
    System.out.println("Сервер запущен на порту " + port);
}

public void run() {
    while (running) {
        try {
            Socket clientSocket = serverSocket.accept();
            String clientIP =
clientSocket.getInetAddress().getHostAddress();
            System.out.println("Клиент подключён: " + clientIP);

            ClientHandler handler = new ClientHandler(clientSocket,
clientIP, userManager, clients);

            synchronized (clients) {
                clients.add(handler);
            }

            Thread clientThread = new Thread(handler);
            clientThread.setDaemon(true);
            clientThread.start();

        } catch (IOException e) {
            if (running) {
                System.err.println("Ошибка принятия подключения: " +
e.getMessage());
            }
        }
    }
}

public void stop() {
    running = false;
    synchronized (clients) {
        for (ClientHandler client : clients) {
            client.disconnect();
        }
        clients.clear();
    }
    try { serverSocket.close(); } catch (IOException ignored) {}
    System.out.println("Сервер остановлен");
}

public static void main(String[] args) {
    try {
        //Устанавливаем кодировку для консоли
        System.setOut(new PrintStream(System.out, true, "UTF-8"));
        System.setErr(new PrintStream(System.err, true, "UTF-8"));

        Server server = new Server(PORT);
        server.run();
    } catch (UnsupportedEncodingException e) {
        e.printStackTrace();
    } catch (IOException e) {
        System.err.println("Server startup error: " + e.getMessage());
    }
}
}

```

UserManager.java:

```
package server;

import java.io.*;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

//Менеджер пользователей: регистрация, вход, хранение данных
public class UserManager {
    private final ConcurrentHashMap<String, String> users; // логин -> пароль
    private final String userFile = "users.dat";
    private final Object fileLock = new Object();

    public UserManager() {
        this.users = new ConcurrentHashMap<>();
        loadUsersFromFile();
    }

    //Проверка логина и пароля
    public boolean validateLogin(String login, String password) {
        String storedPassword = users.get(login);
        return storedPassword != null && storedPassword.equals(password);
    }

    //Регистрация нового пользователя
    public boolean registerUser(String login, String password) {
        if (users.containsKey(login)) {
            return false;
        }
        users.put(login, password);
        saveUsersToFile();
        return true;
    }

    public List<String> getAllLogins() {
        return new ArrayList<>(users.keySet());
    }

    private void loadUsersFromFile() {
        synchronized (fileLock) {
            File file = new File(userFile);
            if (!file.exists()) return;

            try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
                String line;
                while ((line = reader.readLine()) != null) {
                    int pos = line.indexOf(':');
                    if (pos != -1) {
                        String login = line.substring(0, pos);
                        String password = line.substring(pos + 1);
                        users.put(login, password);
                    }
                }
            } catch (IOException e) {
                System.err.println("Ошибка загрузки пользователей: " + e.getMessage());
            }
        }
    }

    //Сохранение пользователей в файл
    private void saveUsersToFile() {
        synchronized (fileLock) {
            try (PrintWriter writer = new PrintWriter(new FileWriter(userFile))) {
                for (Map.Entry<String, String> entry : users.entrySet()) {
                    writer.println(entry.getKey() + ":" + entry.getValue());
                }
            } catch (IOException e) {
                System.err.println("Ошибка сохранения пользователей: " + e.getMessage());
            }
        }
    }
}
```

Приложение Б. UML – диаграммы

На рисунке 23 представлена UML – диаграмма последовательности.

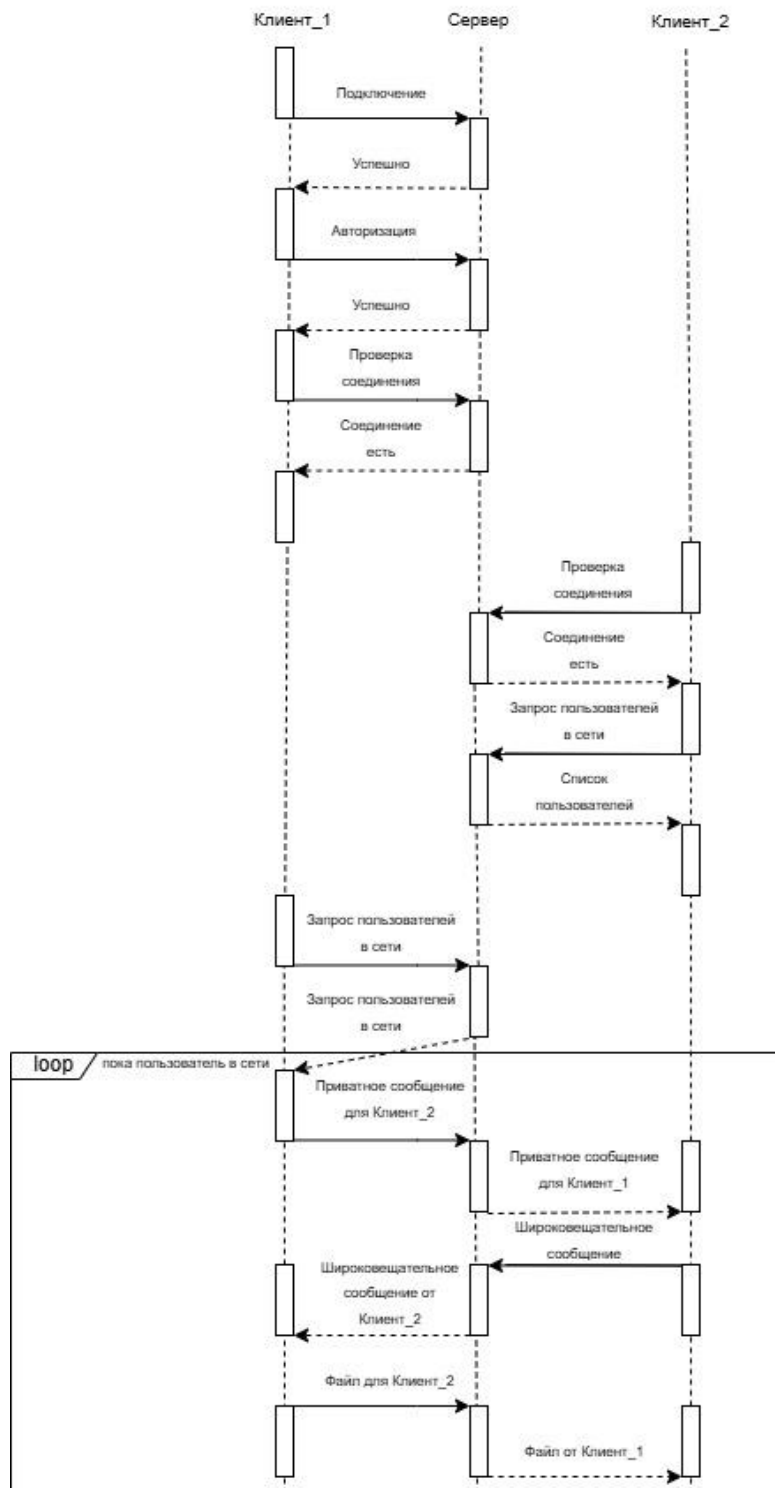


Рисунок 23 – UML – диаграмма последовательности.

На рисунке 24 представлена UML-диаграмма классов.

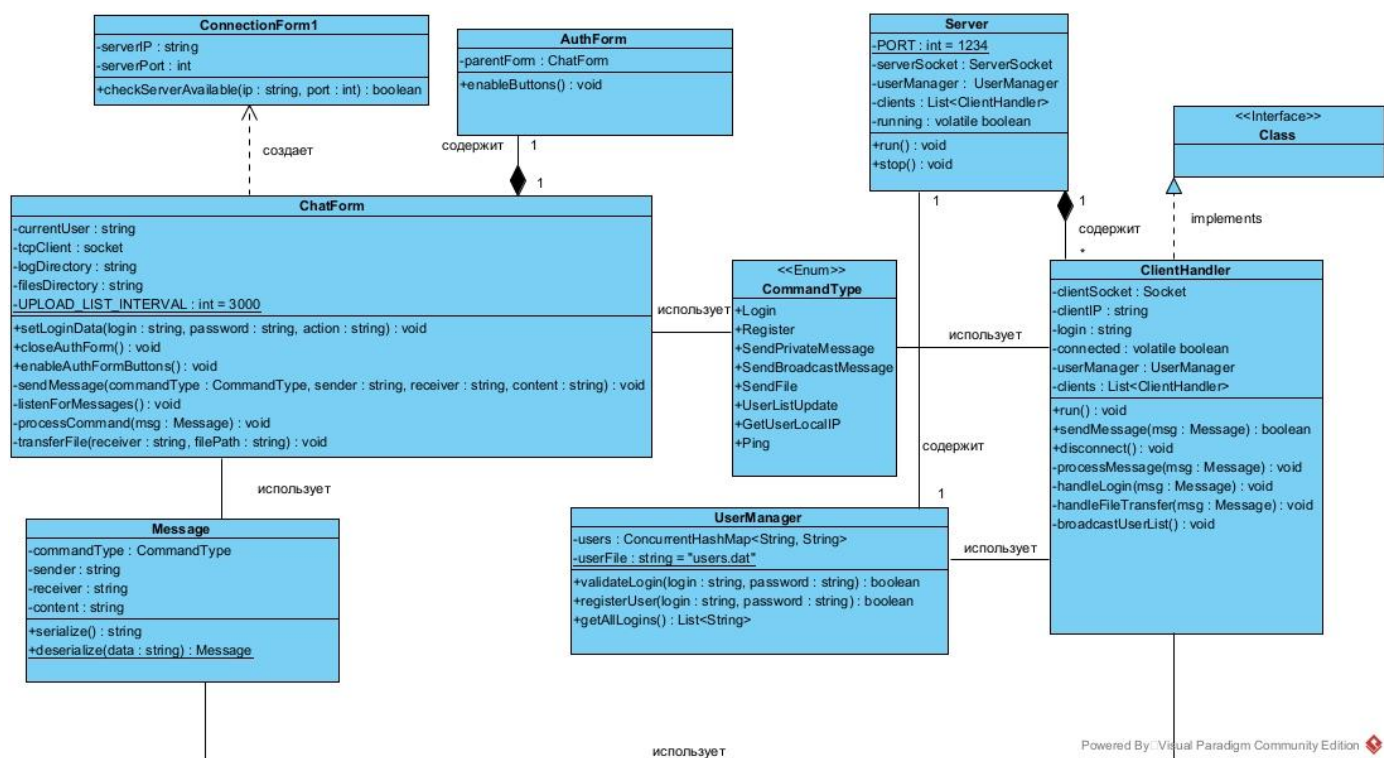


Рисунок 24 – UML-диаграмма классов

На рисунке 25 представлена UML-диаграмма развёртывания.

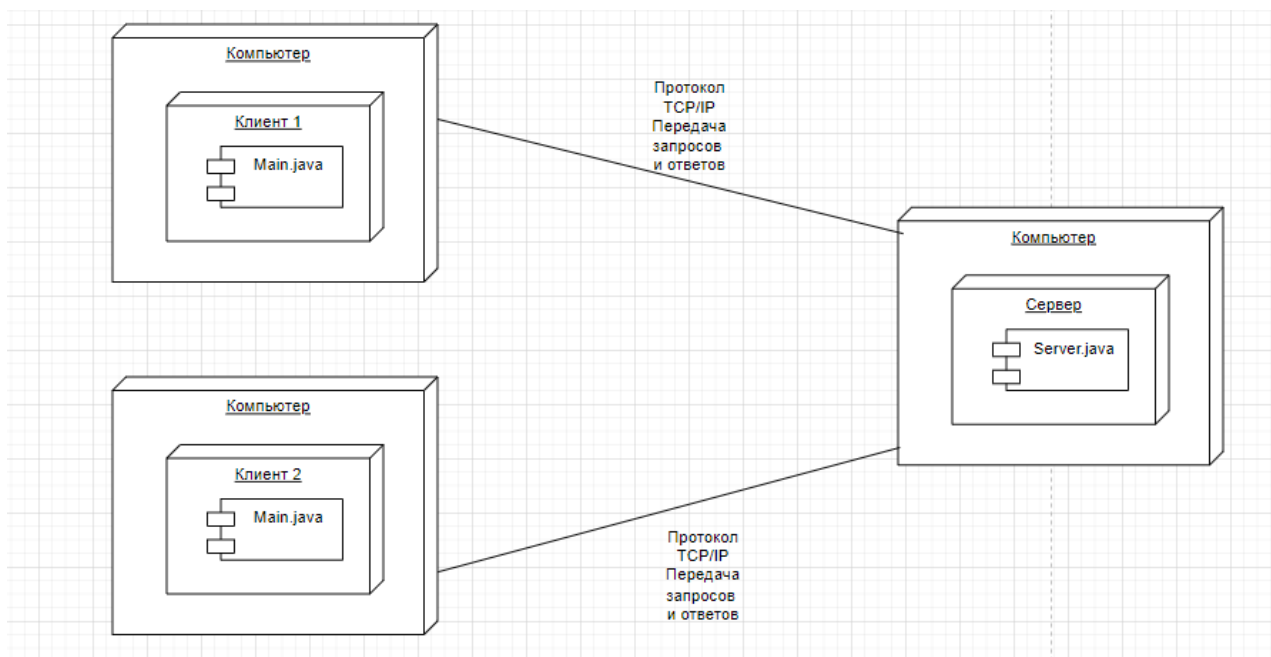


Рисунок 25 – UML-диаграмма развёртывания

На рисунке 26 представлена UML-диаграмма деятельности.

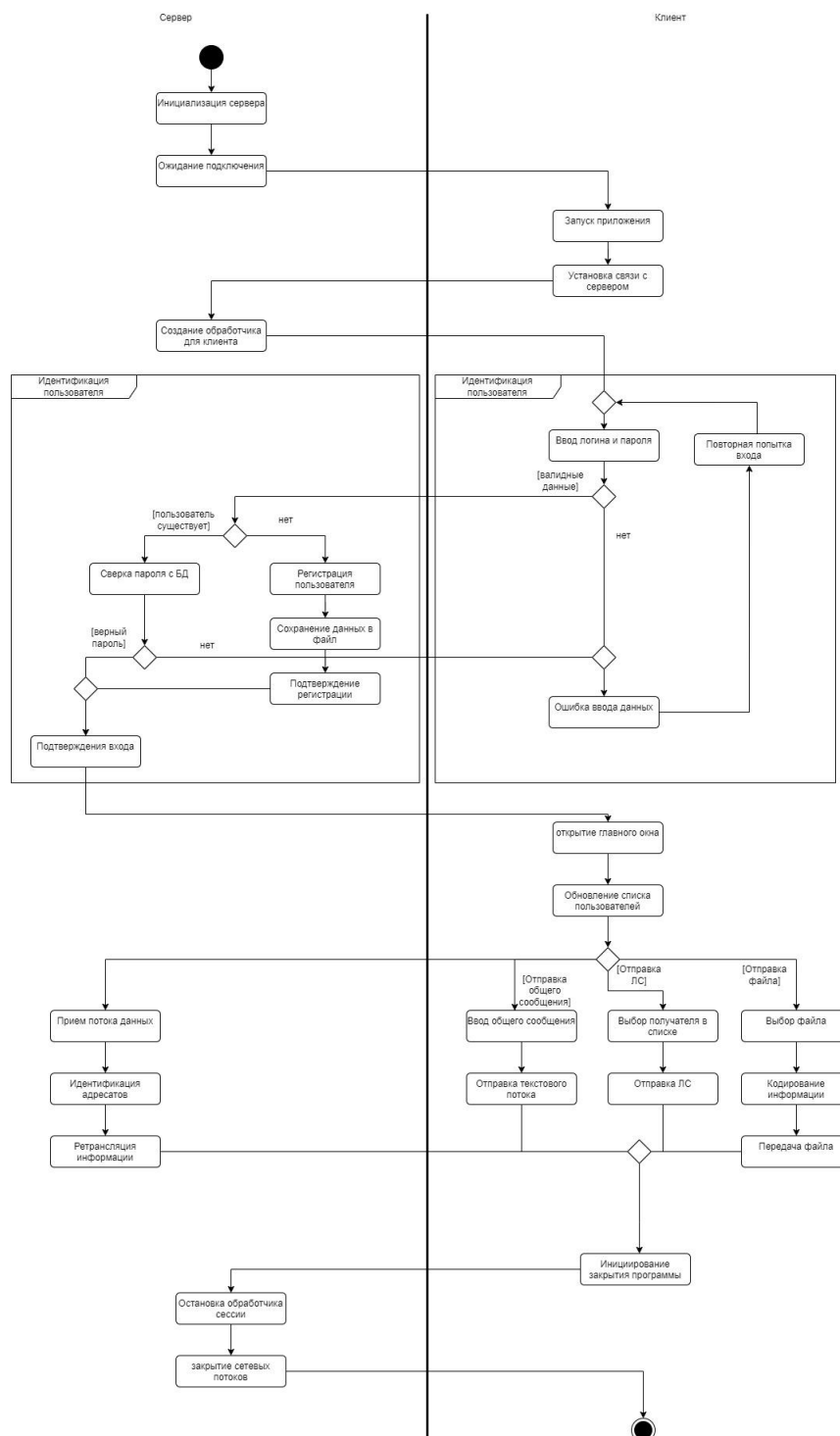


Рисунок 26 – UML-диаграмма деятельности

На рисунке 27 представлена UML-диаграмма вариантов использования.

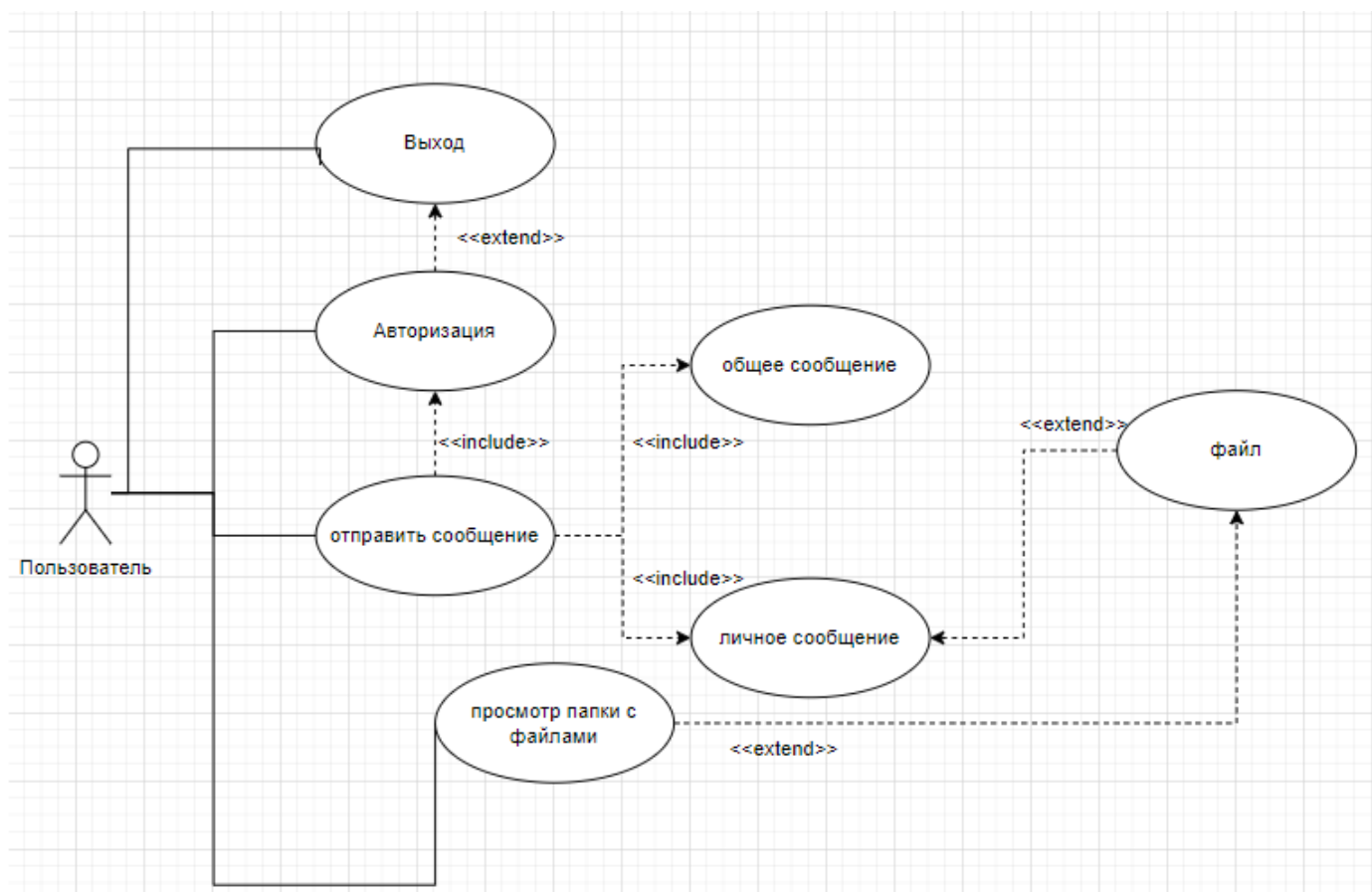


Рисунок 27 – UML-диаграмма вариантов использования